Tecorigin
太初元碁

# Cost Model Analysis & HMM 优化分析

侯银星

优化部



Tecorigin
太初元碁

# 目录


1 算法分析举例

2 单核GEMM

3 多核GEMM

4 复杂约束

Tecorigin
太初元碁

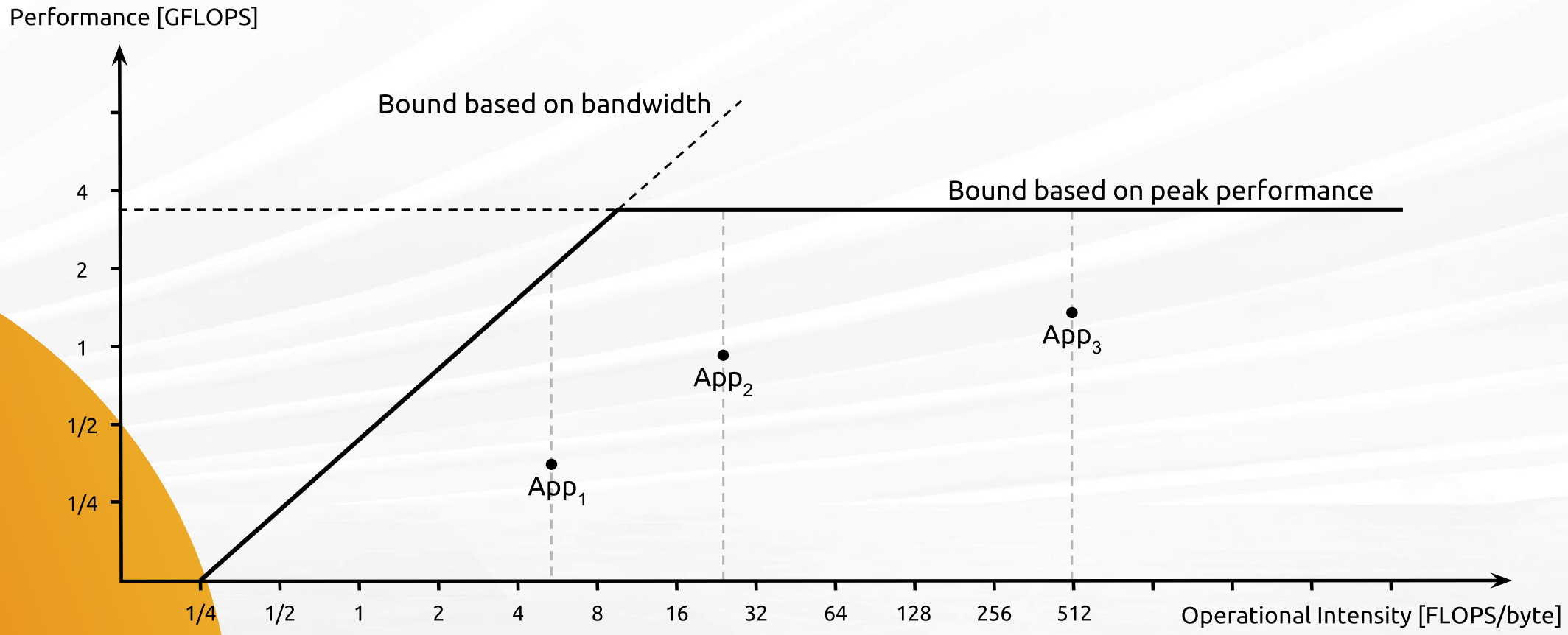
# 1 算法分析

- Roofline Model
- Y = a*X 举例
- PIPELINE
- 性能分析

# Roofline Model

Tecorigin 太初元碁

https://people.eecs.berkeley.edu/~kubitron/cs252/handouts/papers/RooflineVyNoYellow.pdf

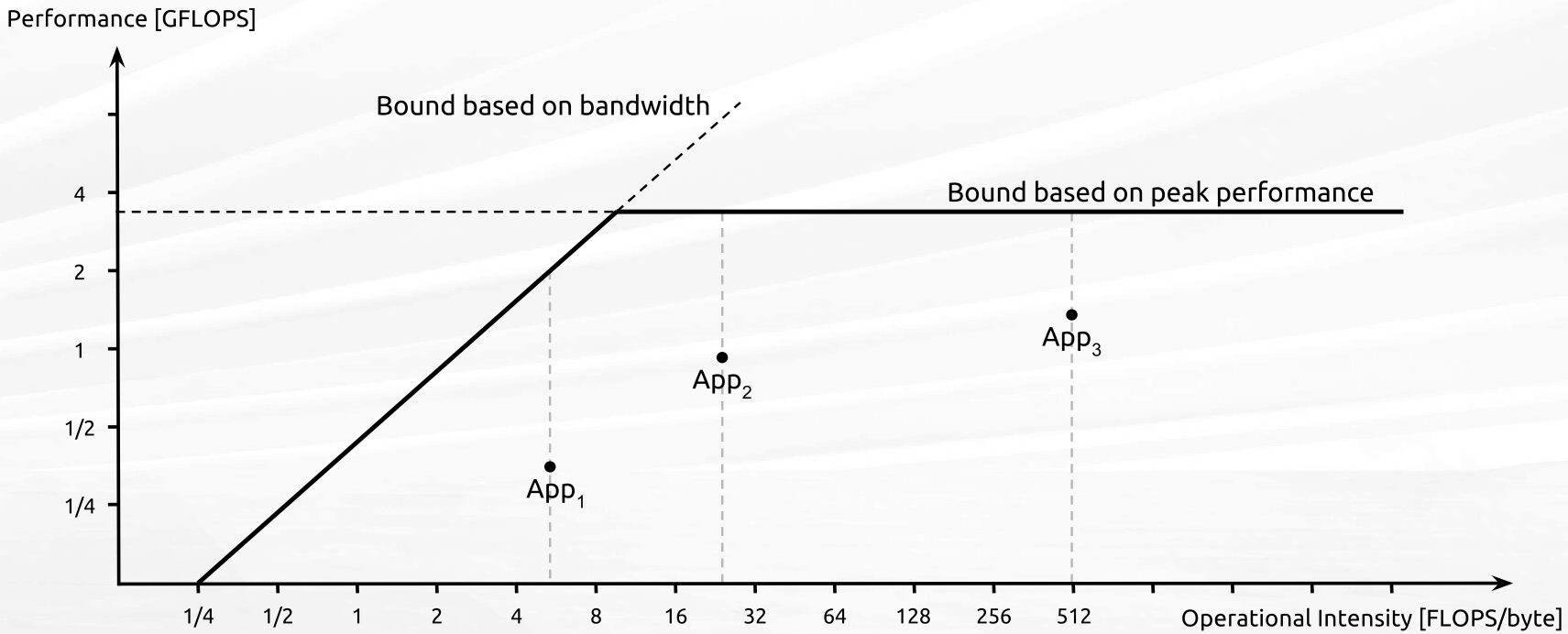



算法分析

# Roofline Model

Tecorigin 太初元碁

$$Intensity_{op} = \frac{Performance}{Bandwidth}$$

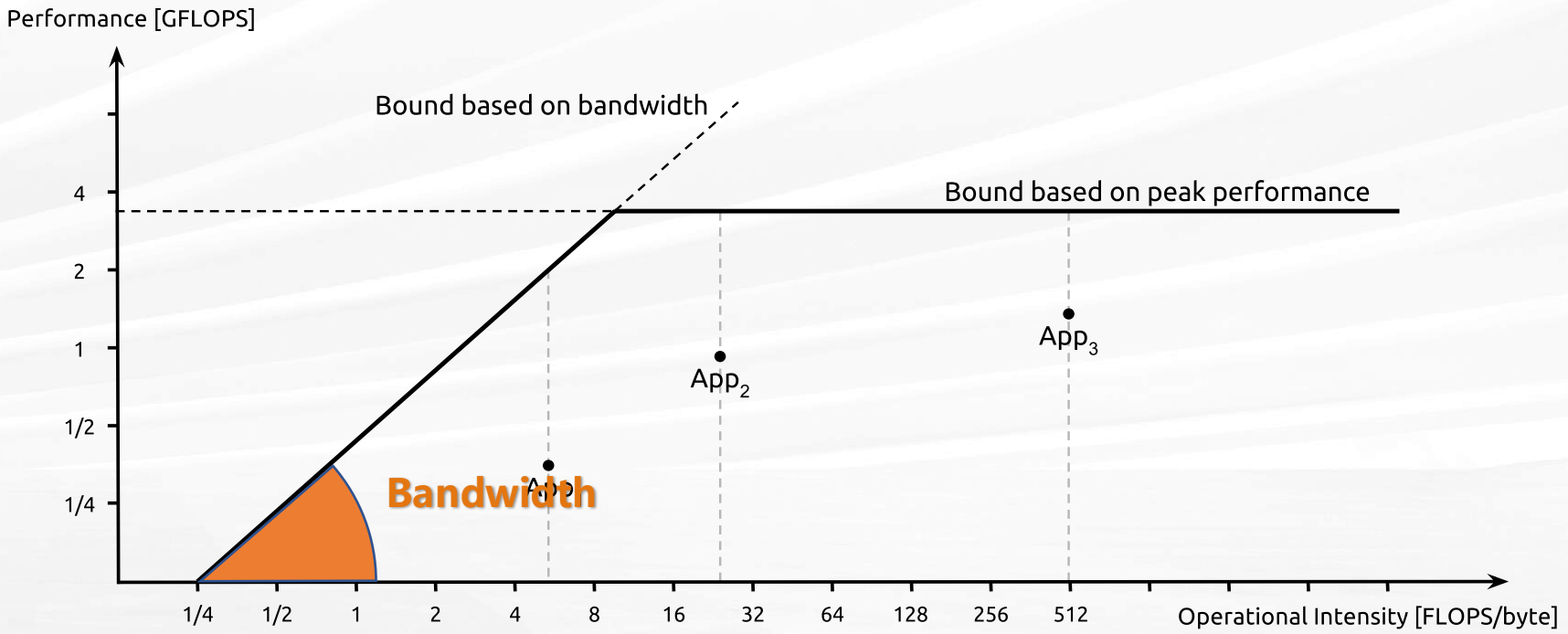



算法分析

# Roofline Model

Tecorigin 太初元碁

$$Intensity_{op} = \frac{Performance}{Bandwidth} \longrightarrow Bandwidth = \frac{Performance}{Intensity_{op}}$$




算法分析

# Y = a*X 举例

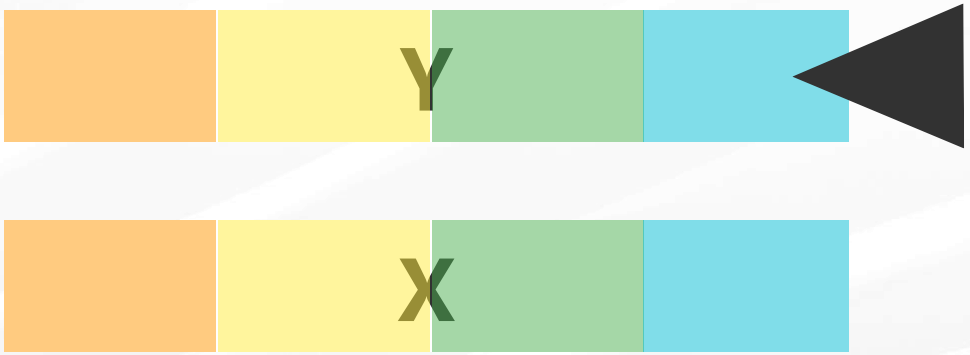
Tecorigin
太初元碁

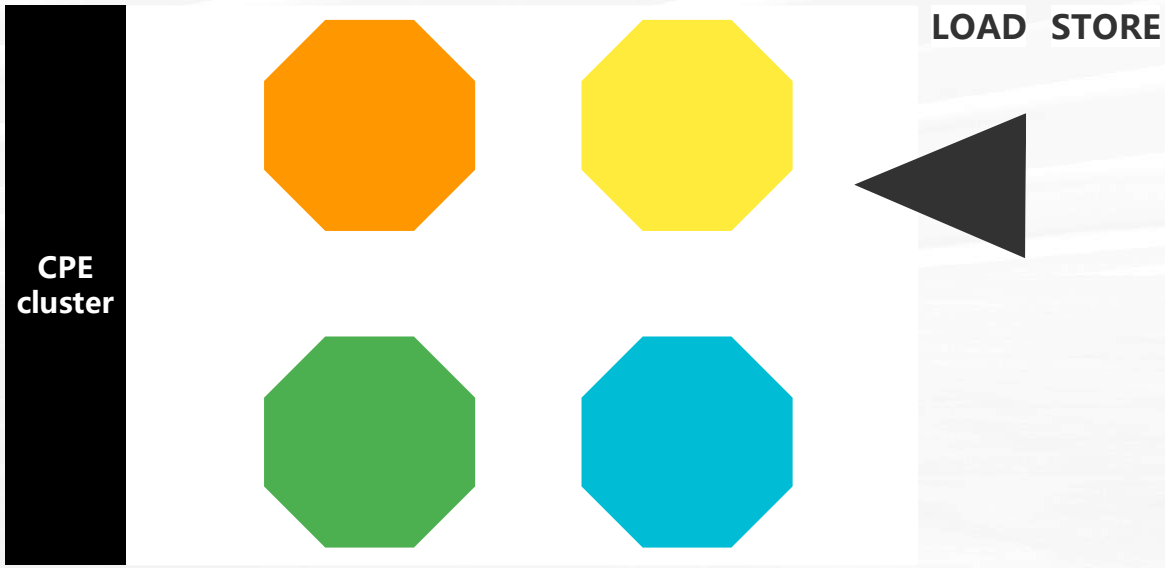
$$Y = alpha * X$$

算法
分析

# Y = a*X 举例

$$Y = alpha * X$$




算法分析

# Y = a*X 举例

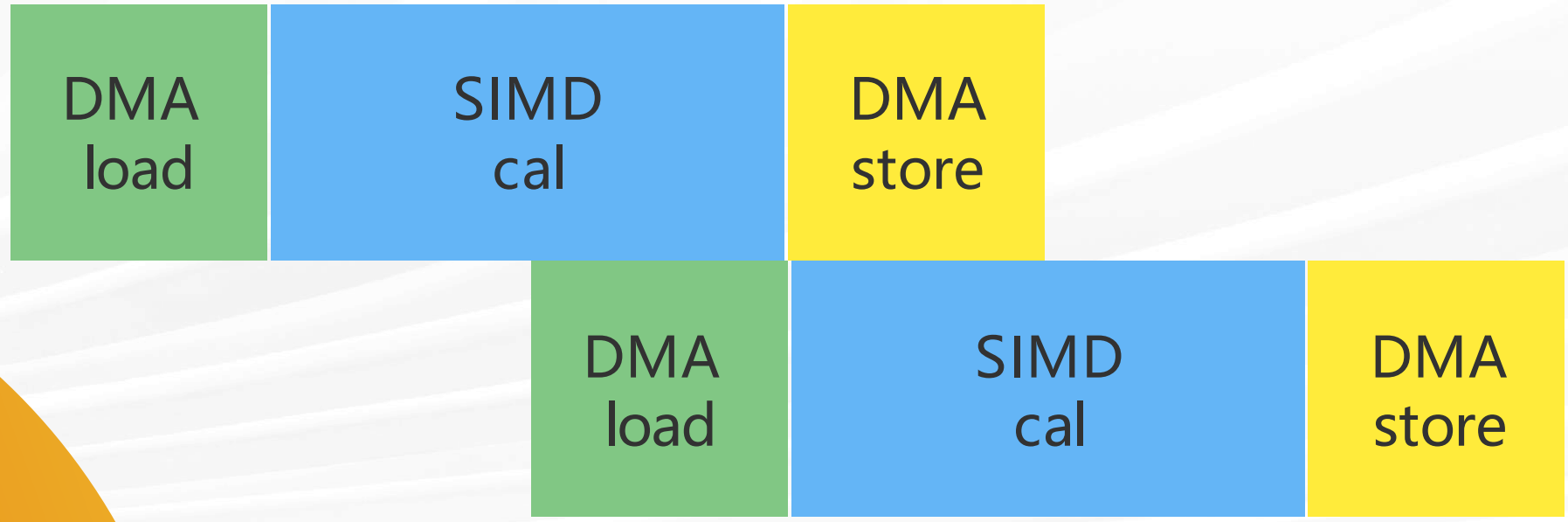
$$Y = alpha * X$$

```
for i in range(P_start, P_end):
        read X[i]
        SIMD Y[i] = alpha * X[i]
        write X[i]
```

算法分析

# PIPELINE

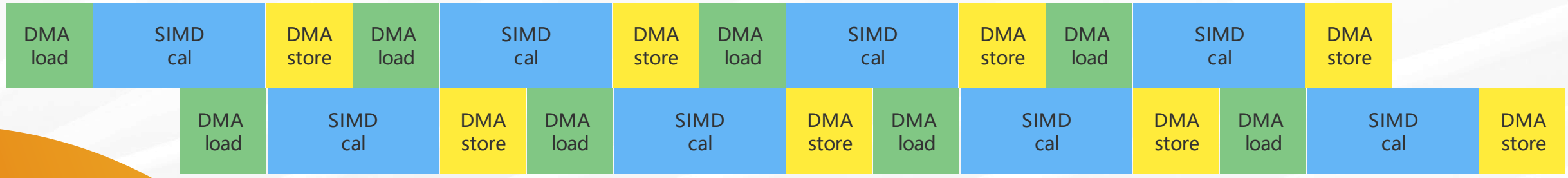
Tecorigin
太初元碁




## 算法分析

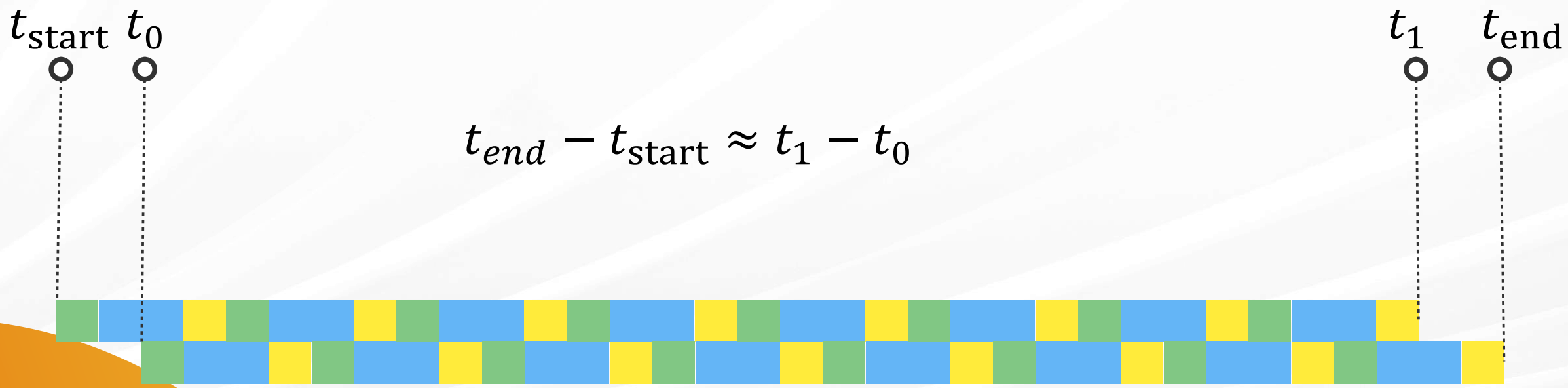
# PIPELINE

Tecorigin
太初元碁




# 算法分析

Tecorigin
太初元碁

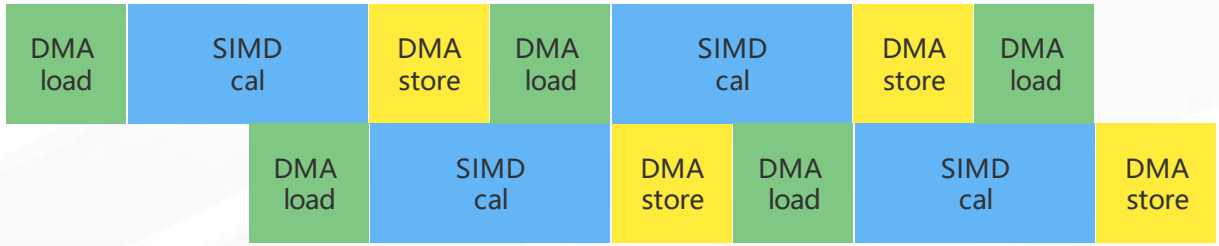


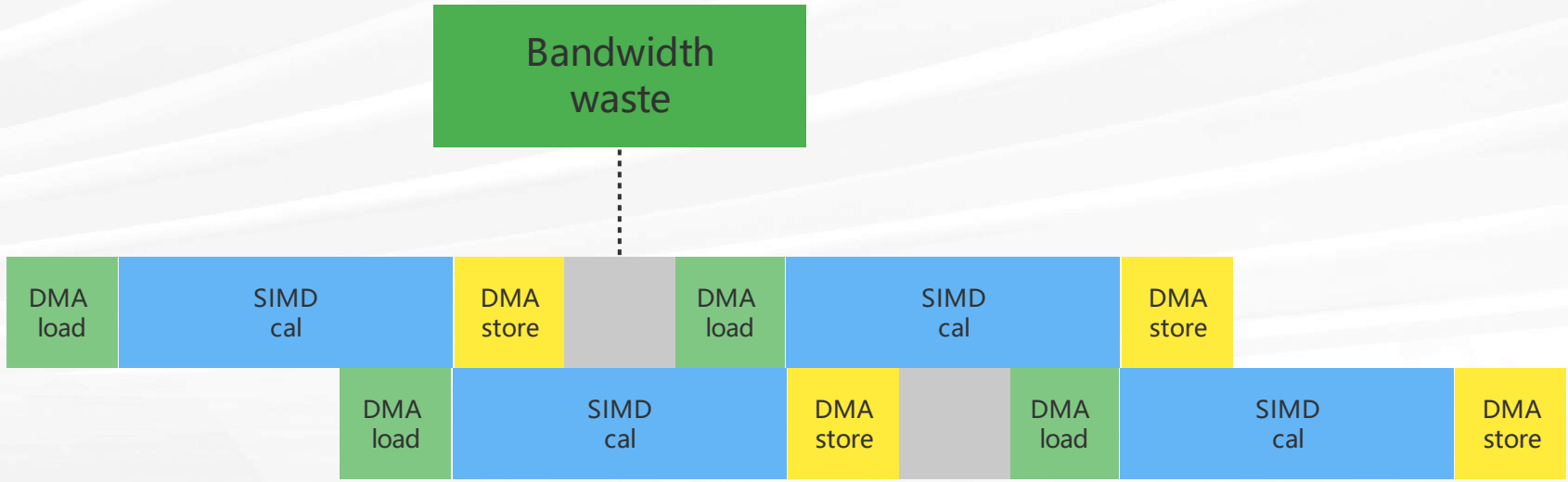

# 算法分析

# PIPELINE

Tecorigin
太初元碁

## 算法分析



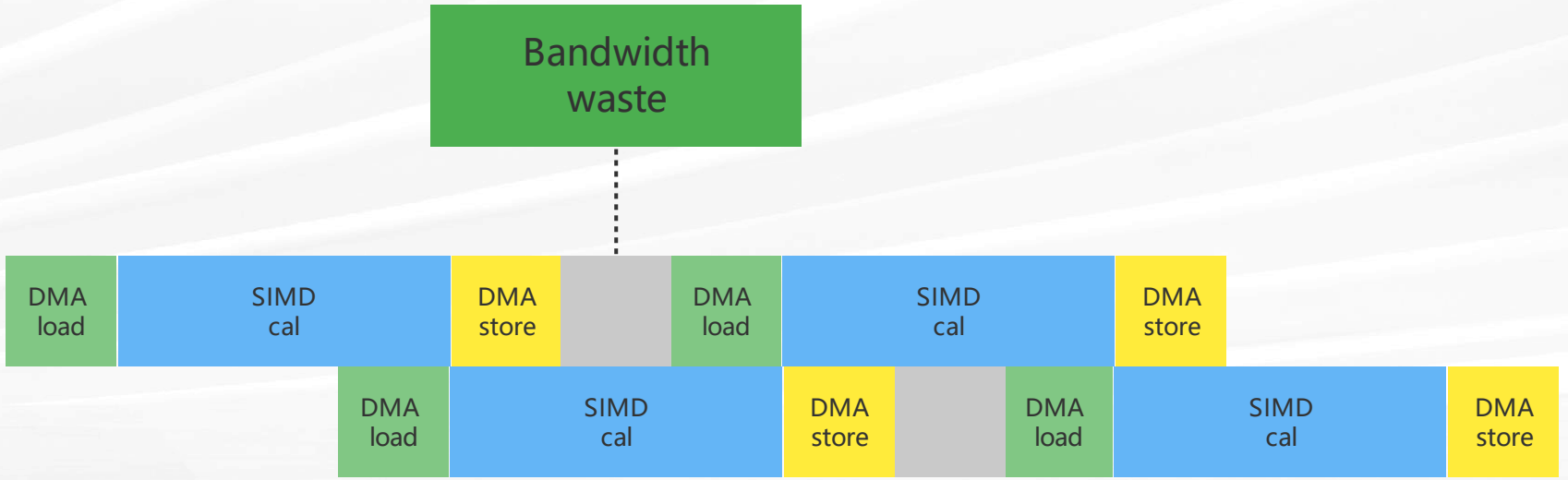
# 性能分析

Tecorigin 太初元碁

$$T_{simd} > T_{load} + T_{store} \longrightarrow T_{simd} > T_{mem}$$

$$\frac{Amount_{simd}}{Performance} > \frac{Amount_{mem}}{Bandwidth} \longrightarrow \frac{Amount_{simd}}{Amount_{mem}} > \frac{Performance}{Bandwidth}$$

## 算法分析

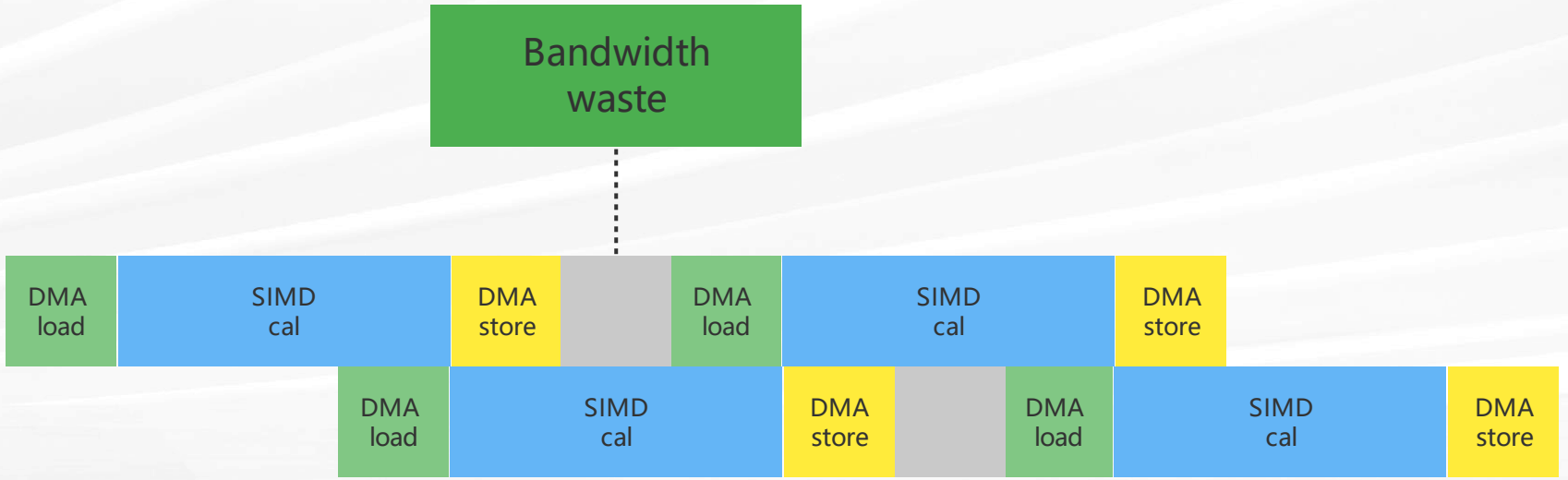


# 性能分析

$$Intensity_{op} = \frac{Performance}{Bandwidth}$$

$$\frac{Amount_{simd}}{Amount_{mem}} > \frac{Performance}{Bandwidth} \rightarrow \frac{Amount_{simd}}{Amount_{mem}} > Intensity_{op}$$

## 算法分析

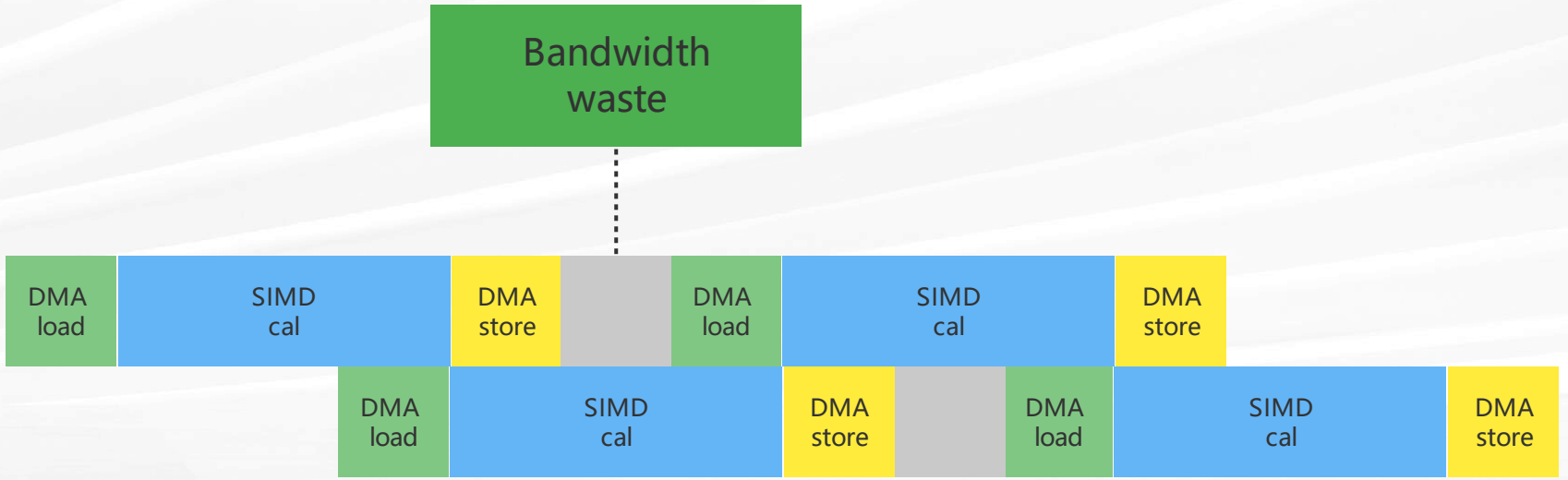


# 性能分析

Tecorigin 太初元碁

$$\frac{Amount_{simd}}{Amount_{mem}} > \frac{Performance}{Bandwidth} \rightarrow \frac{1}{2 * Size_{half}} > \frac{Performance}{Bandwidth}$$

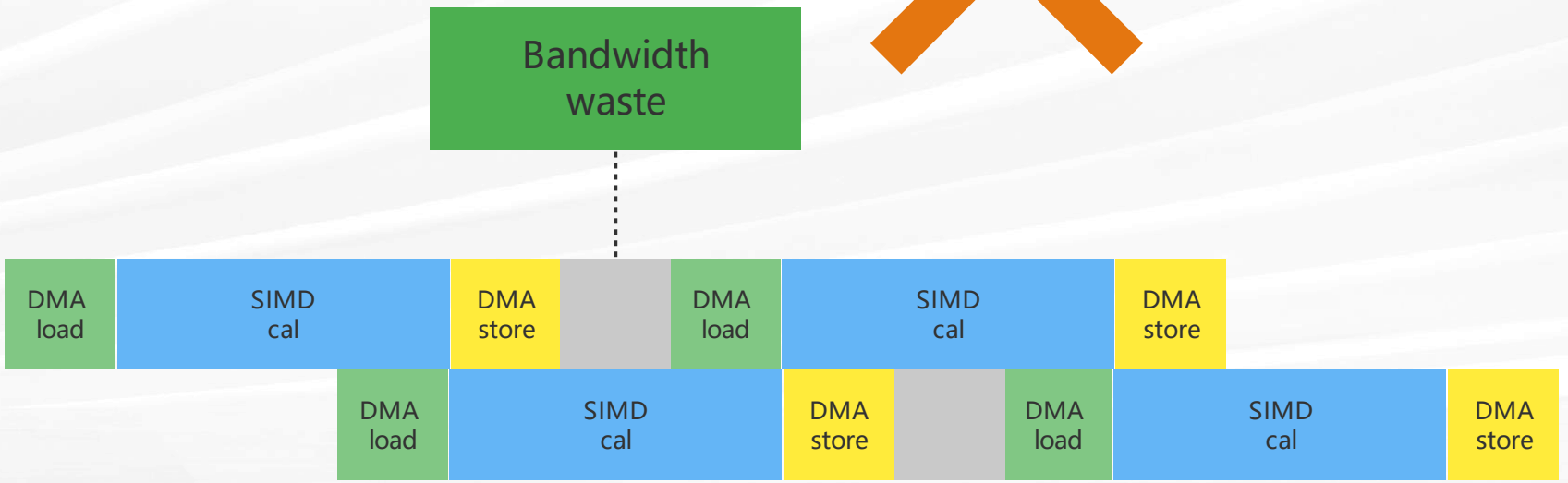
## 算法分析



# 性能分析

$$\frac{1}{Size_{half} * (Num_{read} + Num_{write})} > \frac{Performance}{Bandwidth}$$

$$\frac{1}{4} > \frac{2.5\ Tflops}{250GB/s} \longrightarrow \frac{1}{4} > 10$$



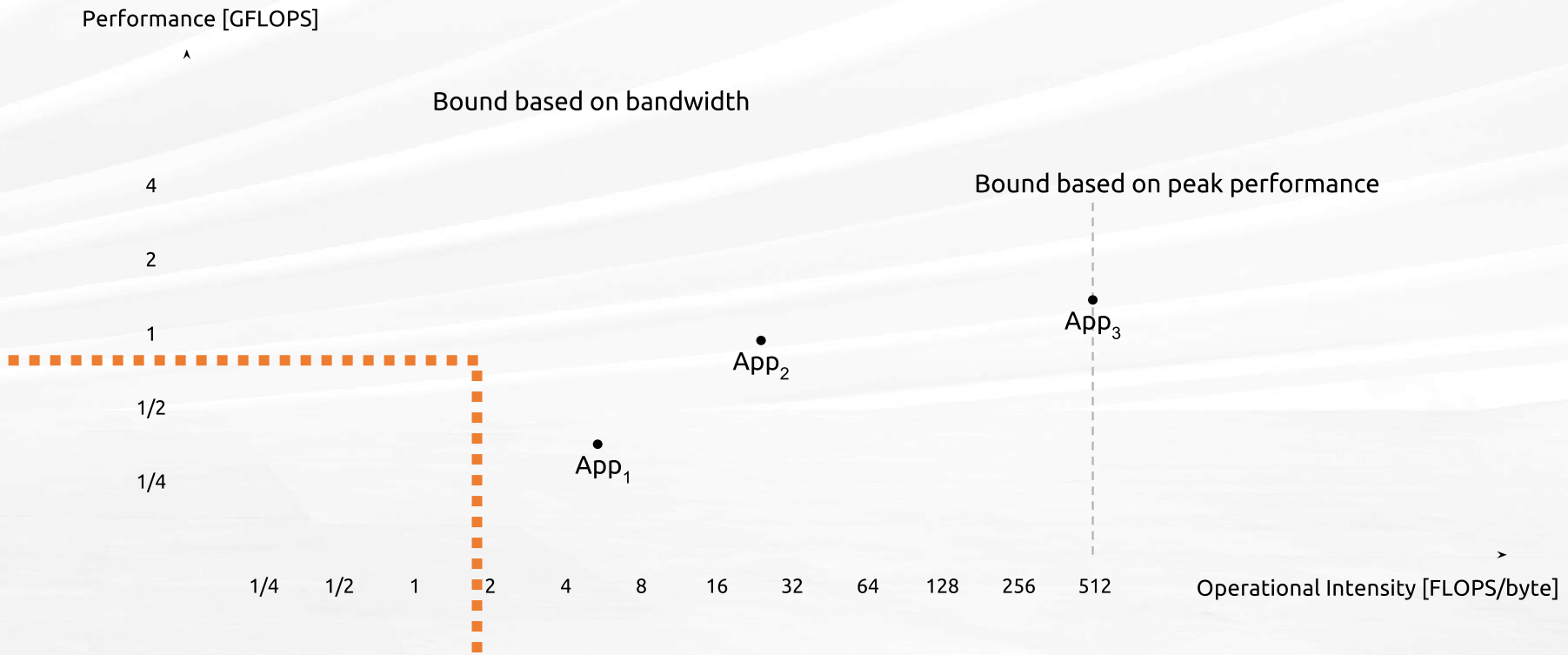

# 算法分析

# 性能分析

Tecorigin 太初元碁

$$\frac{1flop}{4B} > \frac{2.5\ Tflops}{250GB/s}$$

$$\frac{1flop}{4B} * 250GB/_s = 62.5Gflops$$




# 算法分析

Tecorigin
太初元碁

2

# 单核 GEMM

- 存储分级结构
- 一维分块
- 二维分块

# Definition

Tecorigin
太初元碁

$$C = alpha * A * B + beta * C$$

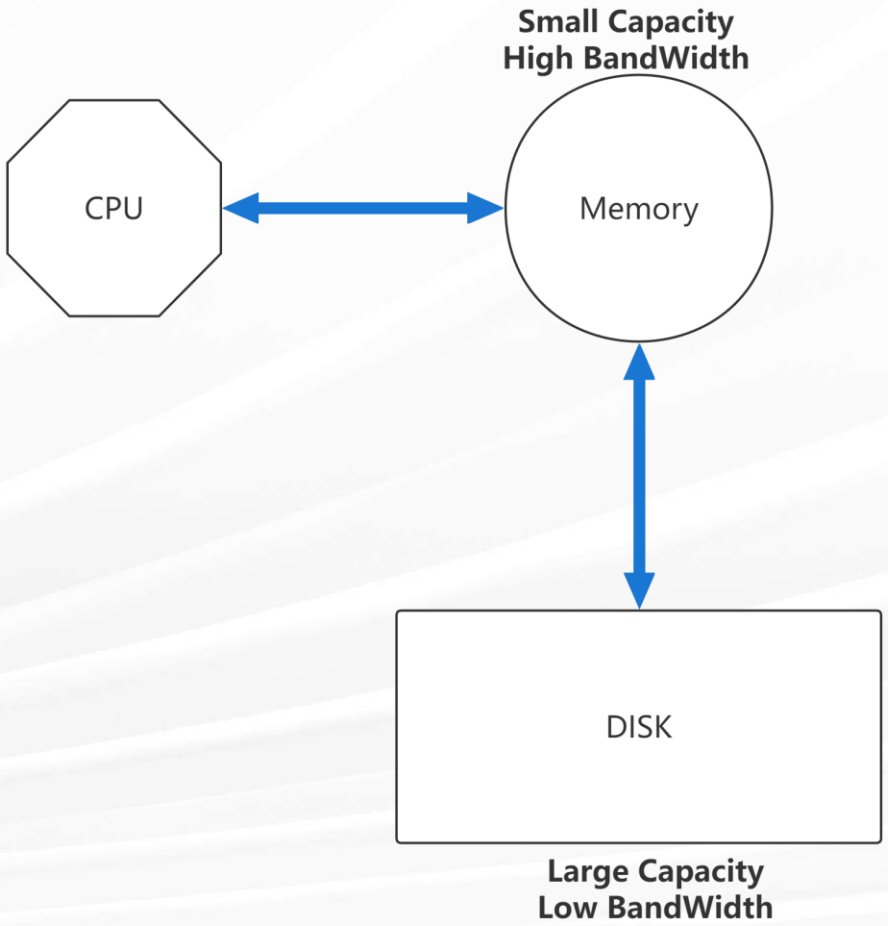
Simplify

$$C = A * B$$

单核
GEMM

# 存储分级结构

Tecorigin
太初元碁

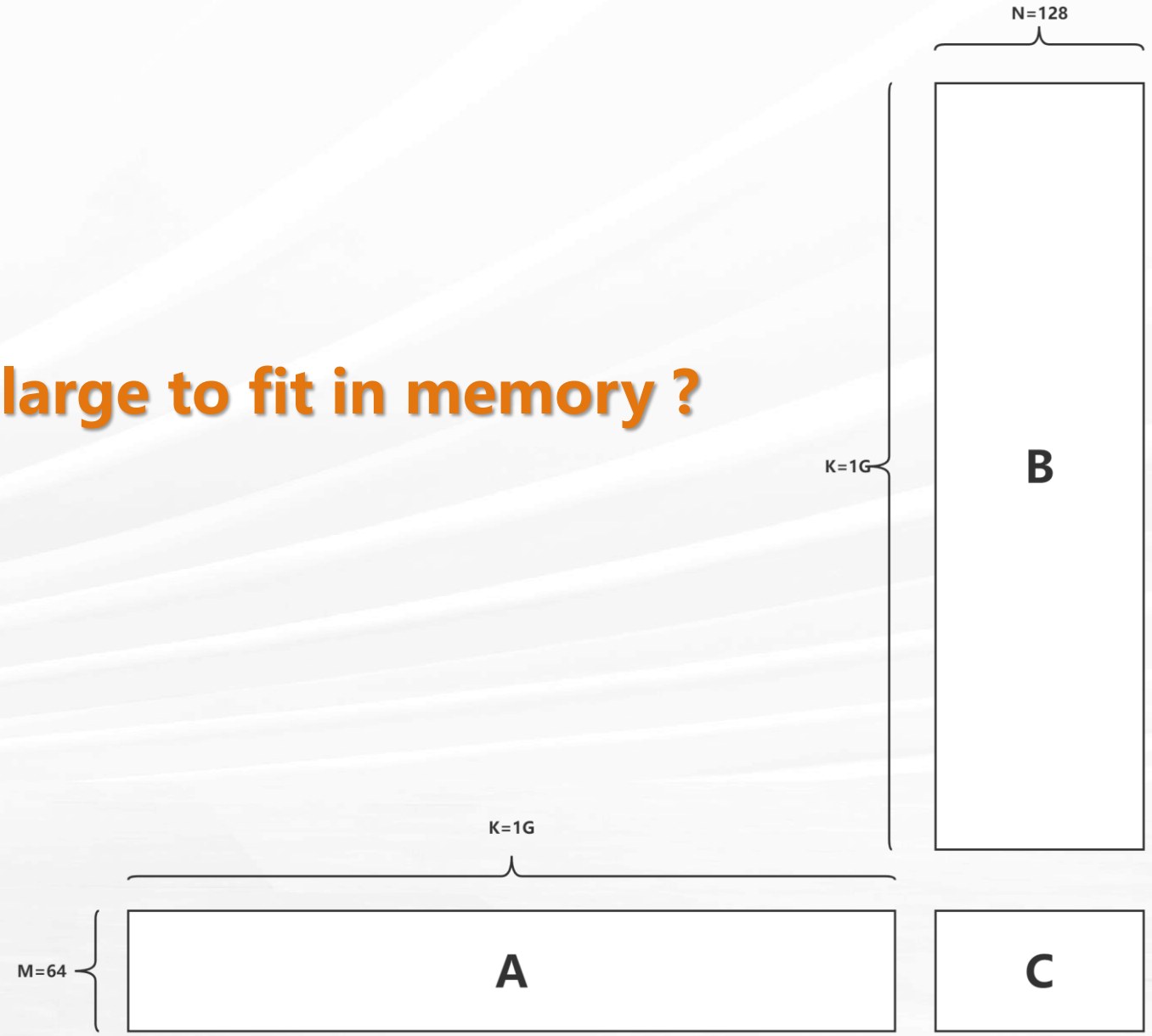



# 单核 GEMM

# 一维分块

Tecorigin 太初元碁

Too large to fit in memory ?



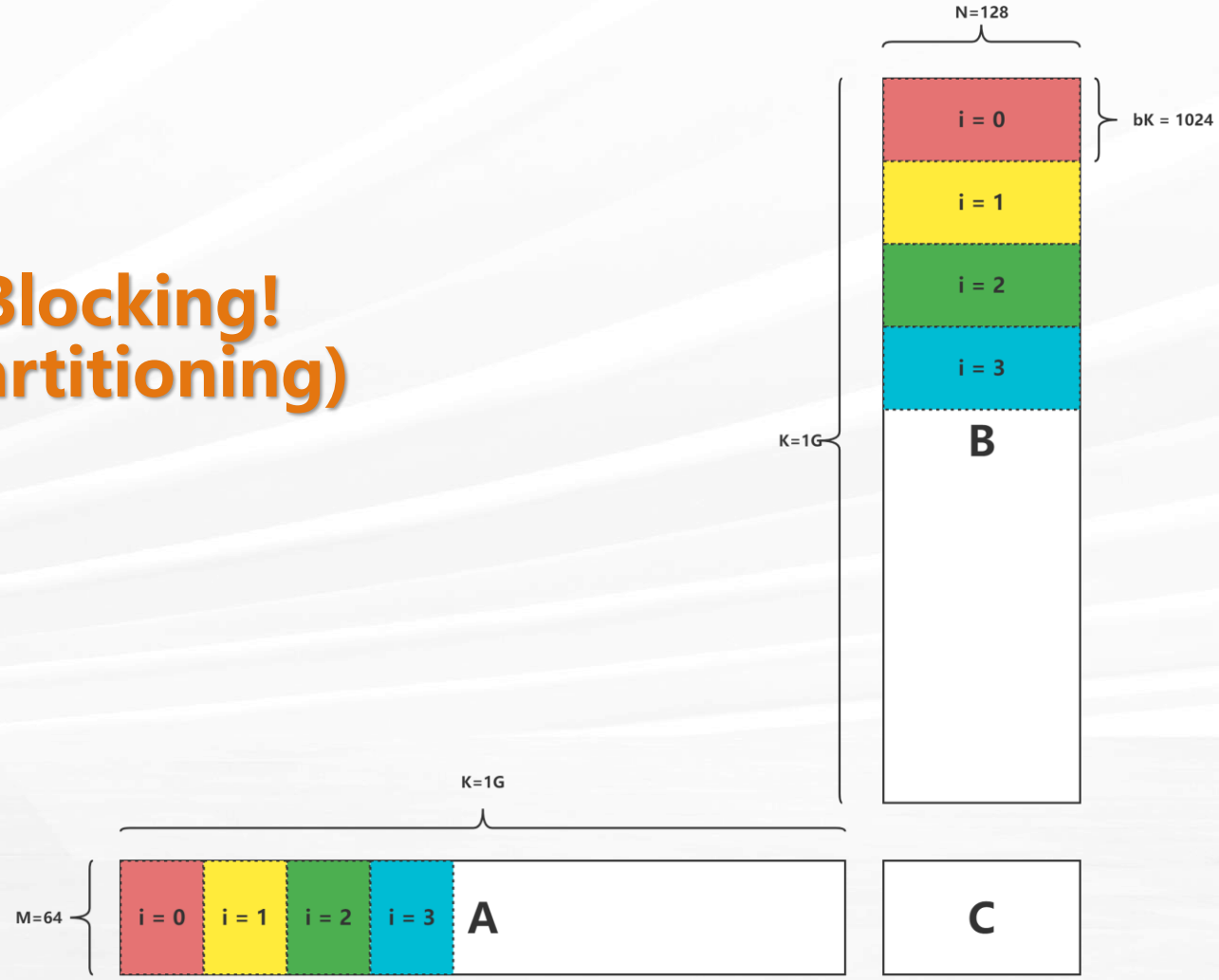

单核 GEMM

# 一维分块

Tecorigin
太初元碁

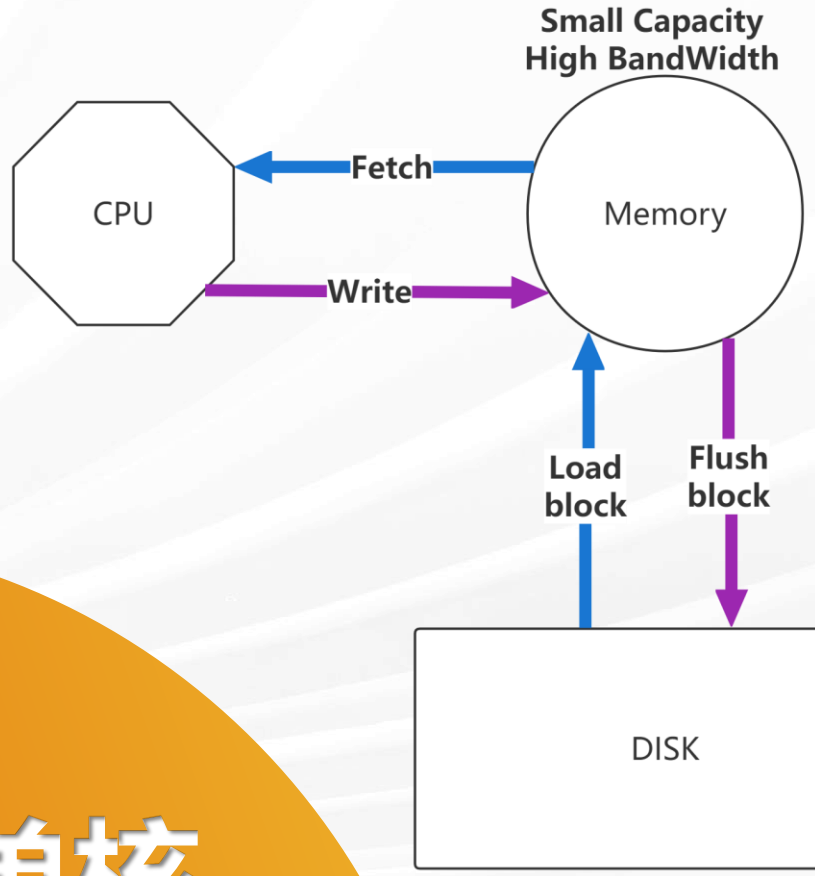
单核
GEMM

**Blocking!**
**(Partitioning)**

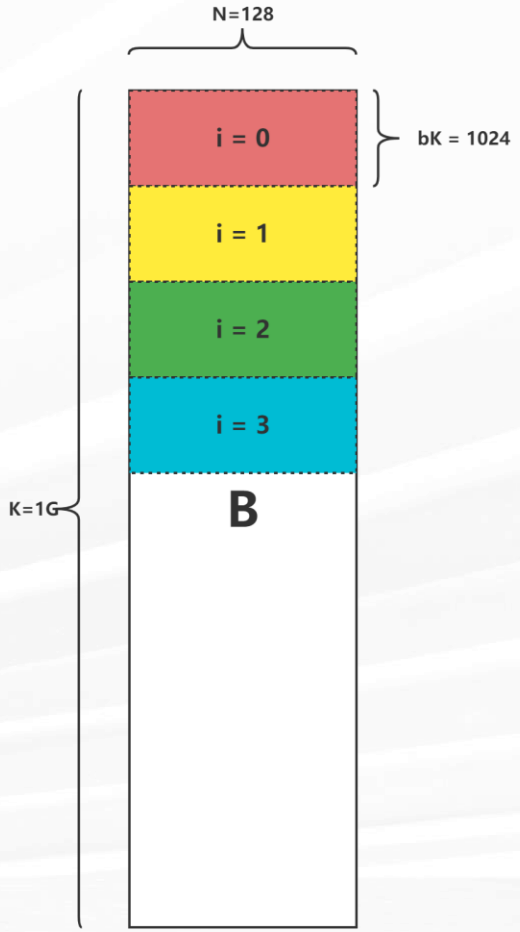


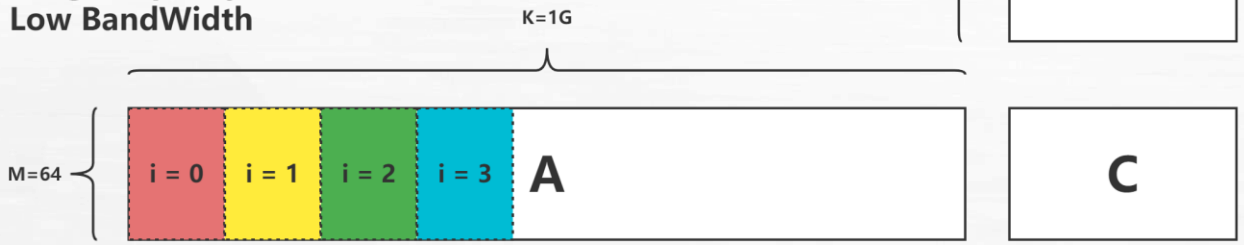
# 一维分块

Tecorigin
太初元碁




**Blocking!**
**(Partitioning)**



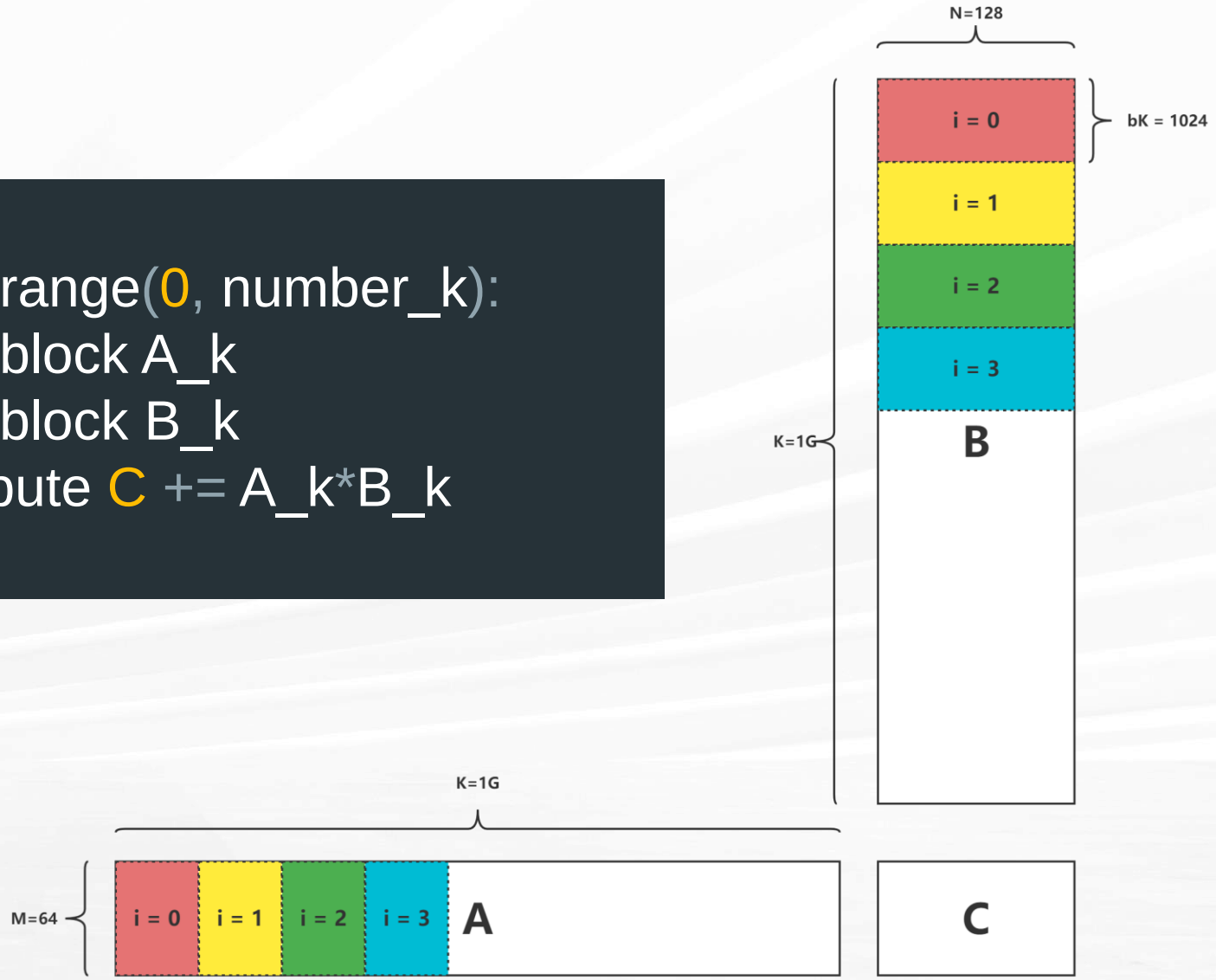




# 单核 GEMM

# 一维分块

Tecorigin 太初元碁

```
load C
for k in range(0, number_k):
    load block A_k
    load block B_k
    compute C += A_k*B_k
store C
```




# 单核 GEMM

# 一维分块

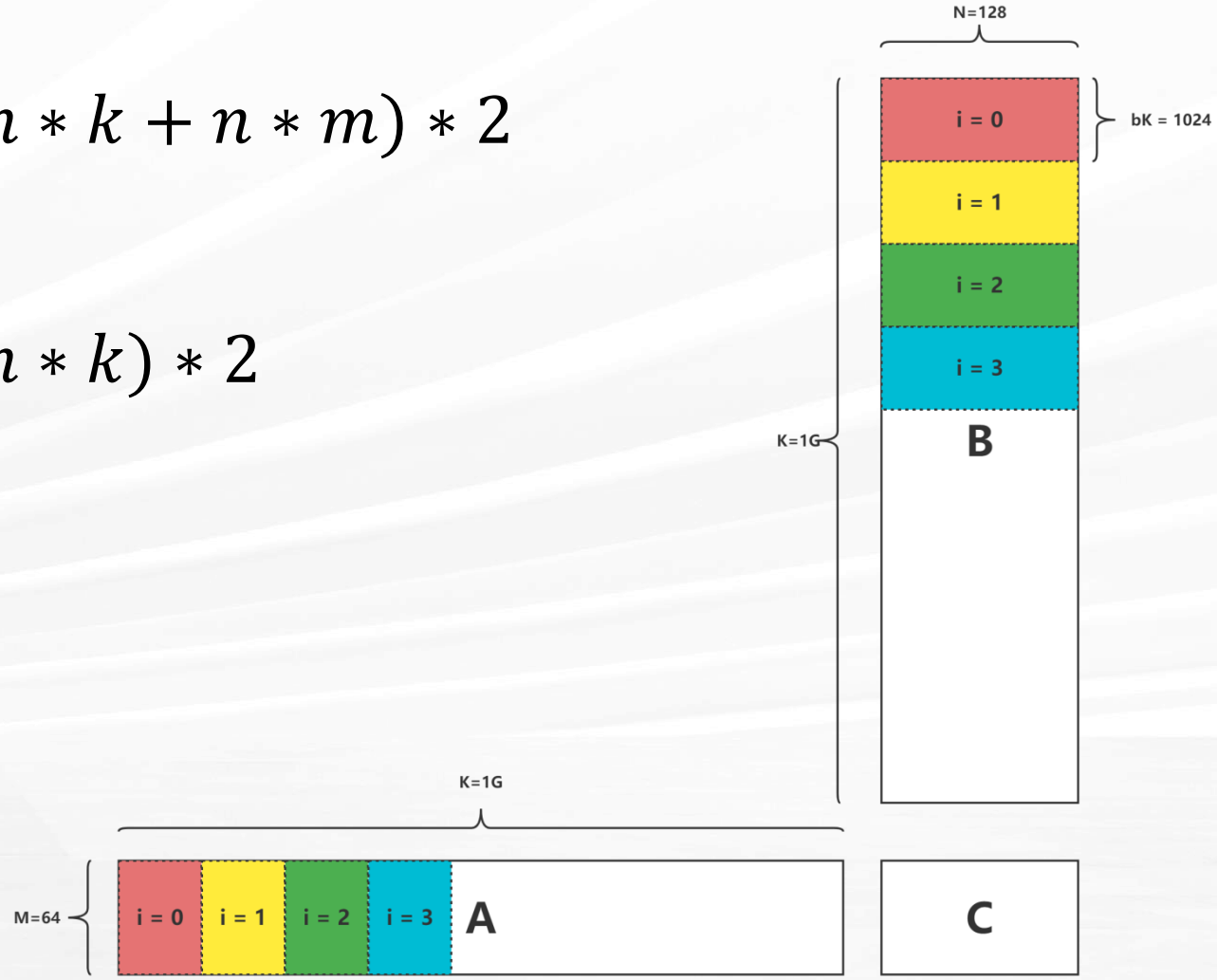
Tecorigin 太初元碁

$$Amount_{mem} = (\mathrm{m} * k + n * k + n * m) * 2$$

**K large enough**

$$Amount_{mem} = (\mathrm{m} * k + n * k) * 2$$

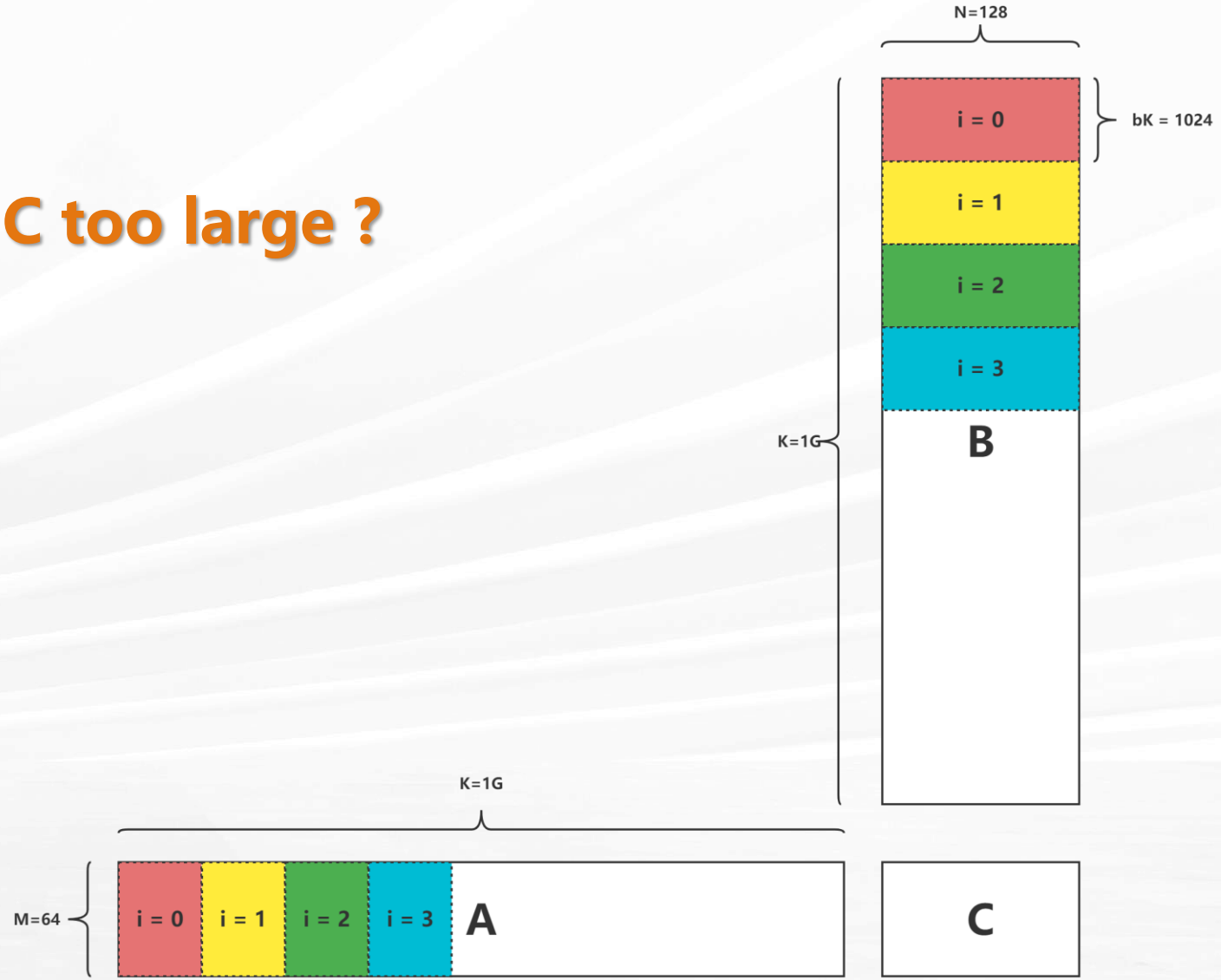
## 单核 GEMM



# 二维分块

Tecorigin 太初元碁

## Size of C too large ?




单核 GEMM

# 二维分块

Tecorigin 太初元碁

## Size of C too large

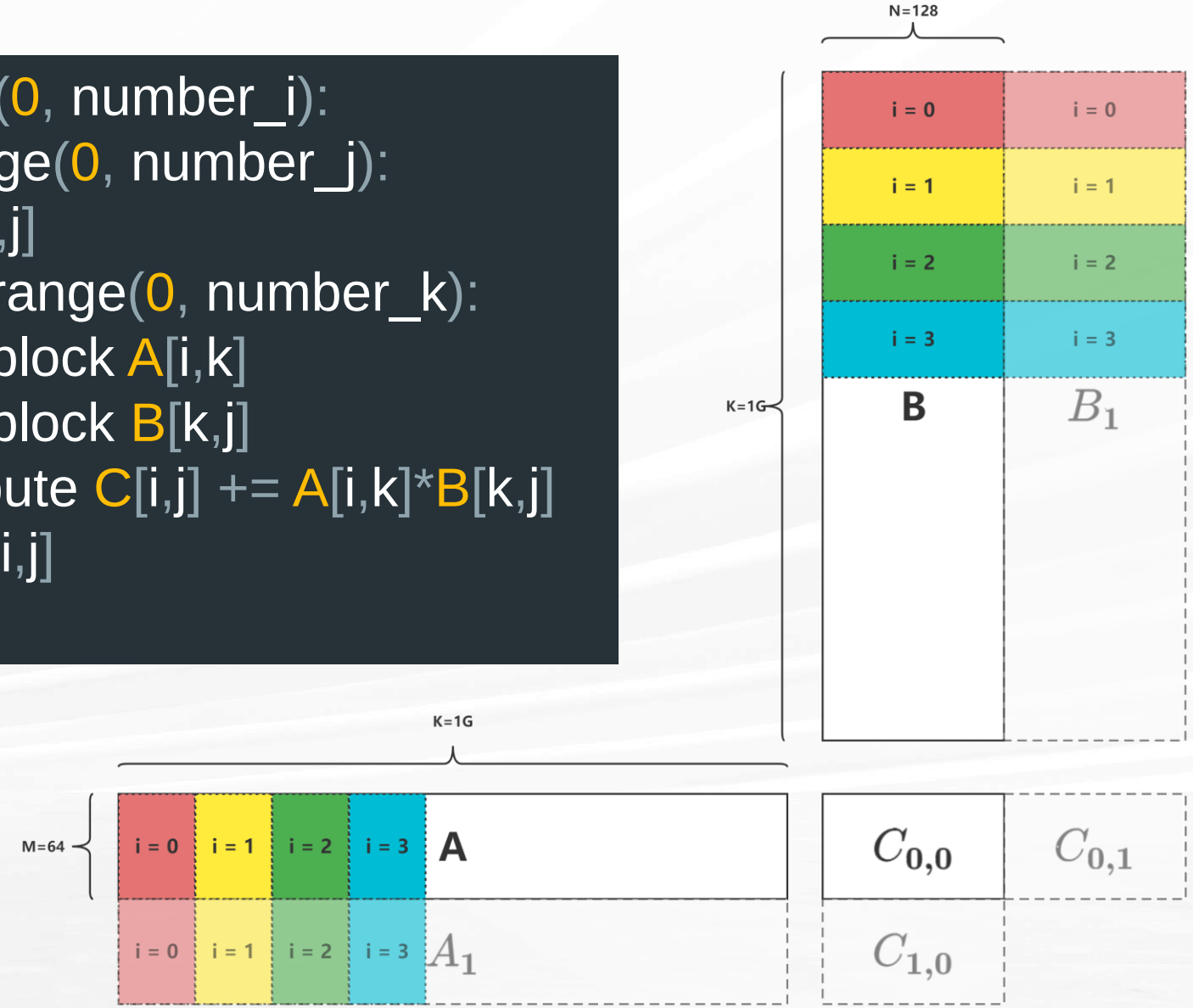



# 单核 GEMM

# 二维分块

Tecorigin 太初元碁

```
for i in range(0, number_i):
    for j in range(0, number_j):
        load C[i,j]
        for k in range(0, number_k):
            load block A[i,k]
            load block B[k,j]
            compute C[i,j] += A[i,k]*B[k,j]
        store C[i,j]
```

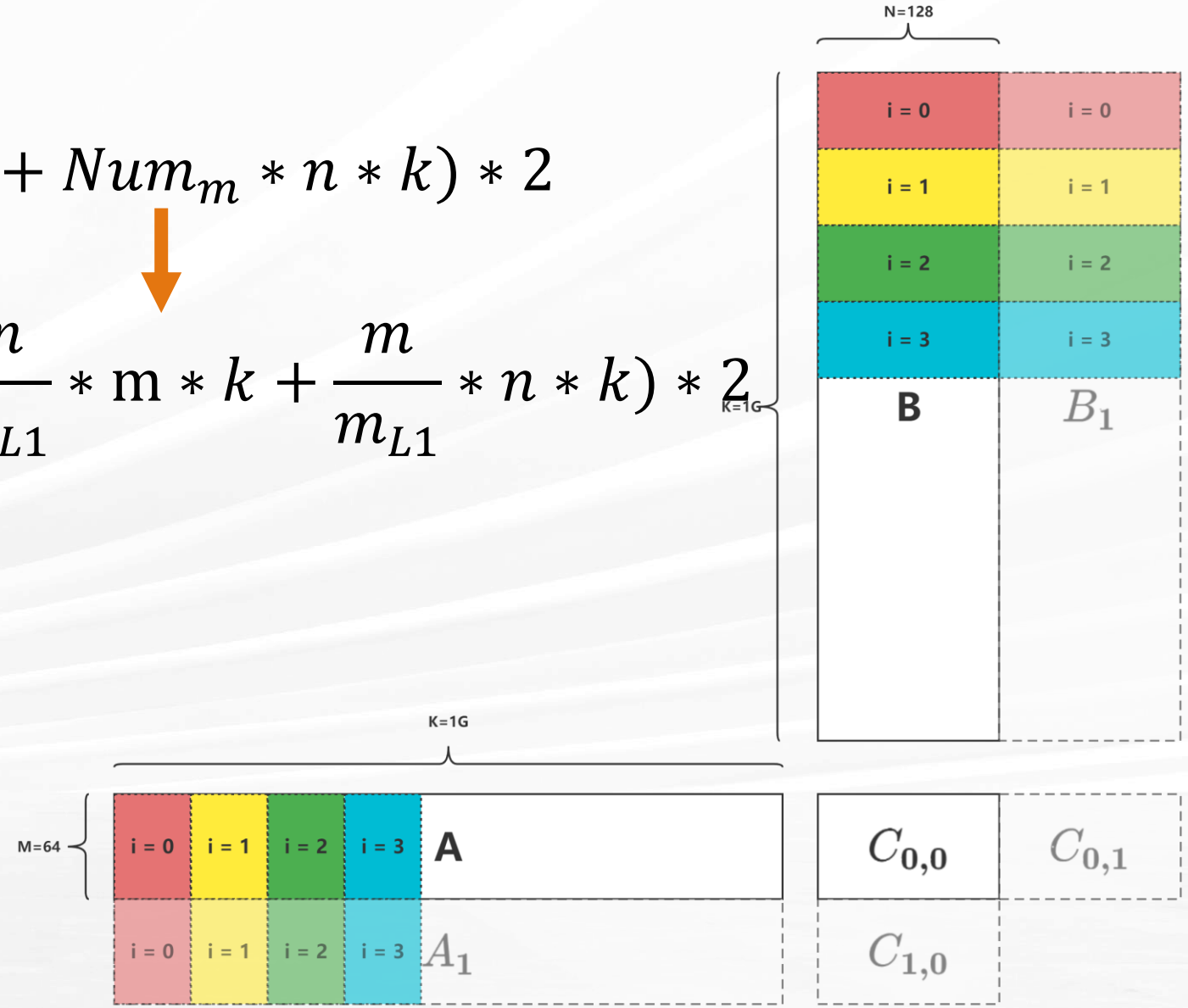



单核 GEMM

# 二维分块

$$Amount_{mem} = (Num_N * \mathrm{m} * k + Num_m * n * k) * 2$$

$$Amount_{mem} = (\frac{n}{n_{L1}} * \mathrm{m} * k + \frac{m}{m_{L1}} * n * k) * 2$$

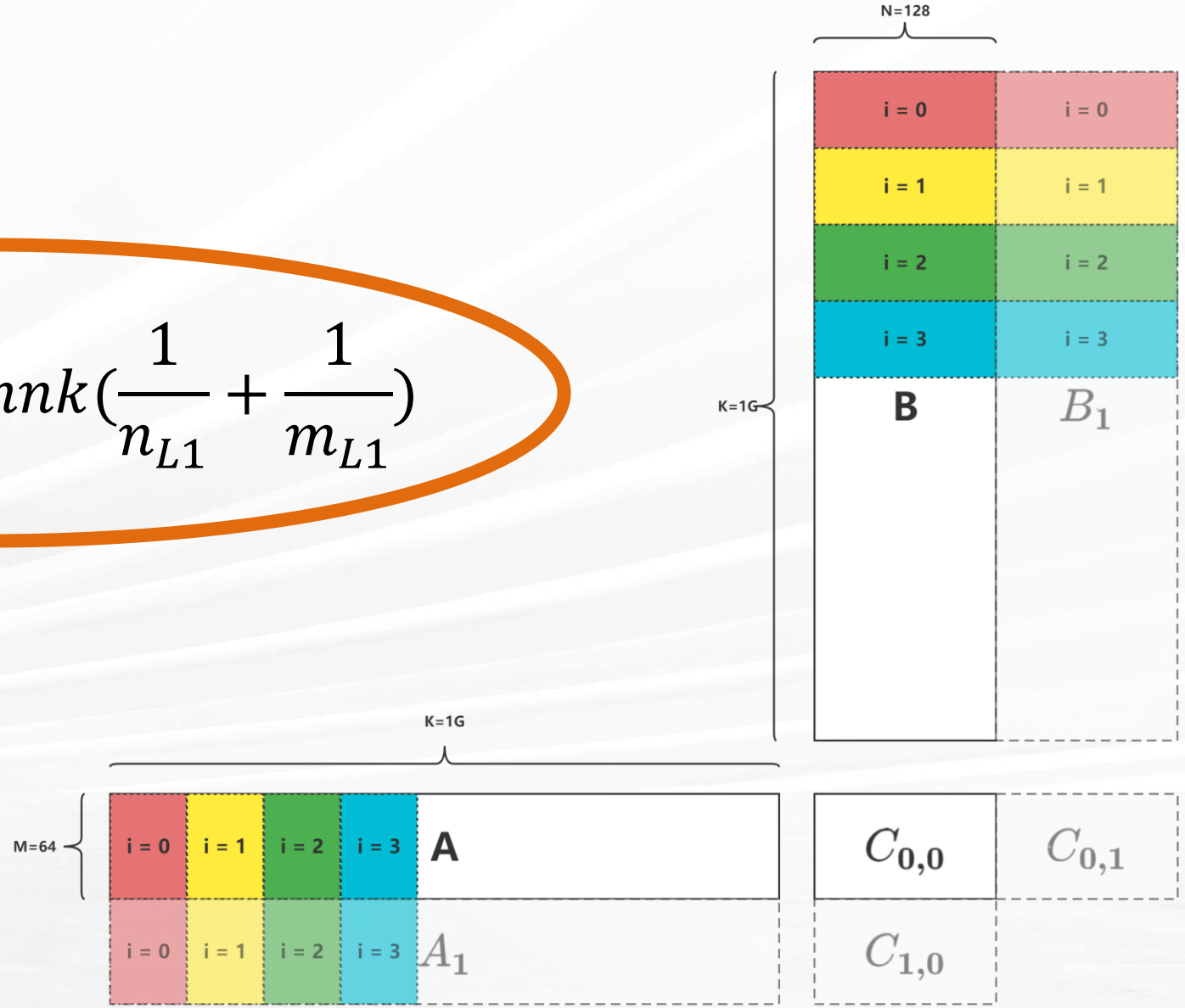



单核 GEMM

# 二维分块

Tecorigin 太初元碁

$$Amount_{mem} = 2mnk(\frac{1}{n_{L1}} + \frac{1}{m_{L1}})$$




# 单核 GEMM

Tecorigin
太初元碁

3

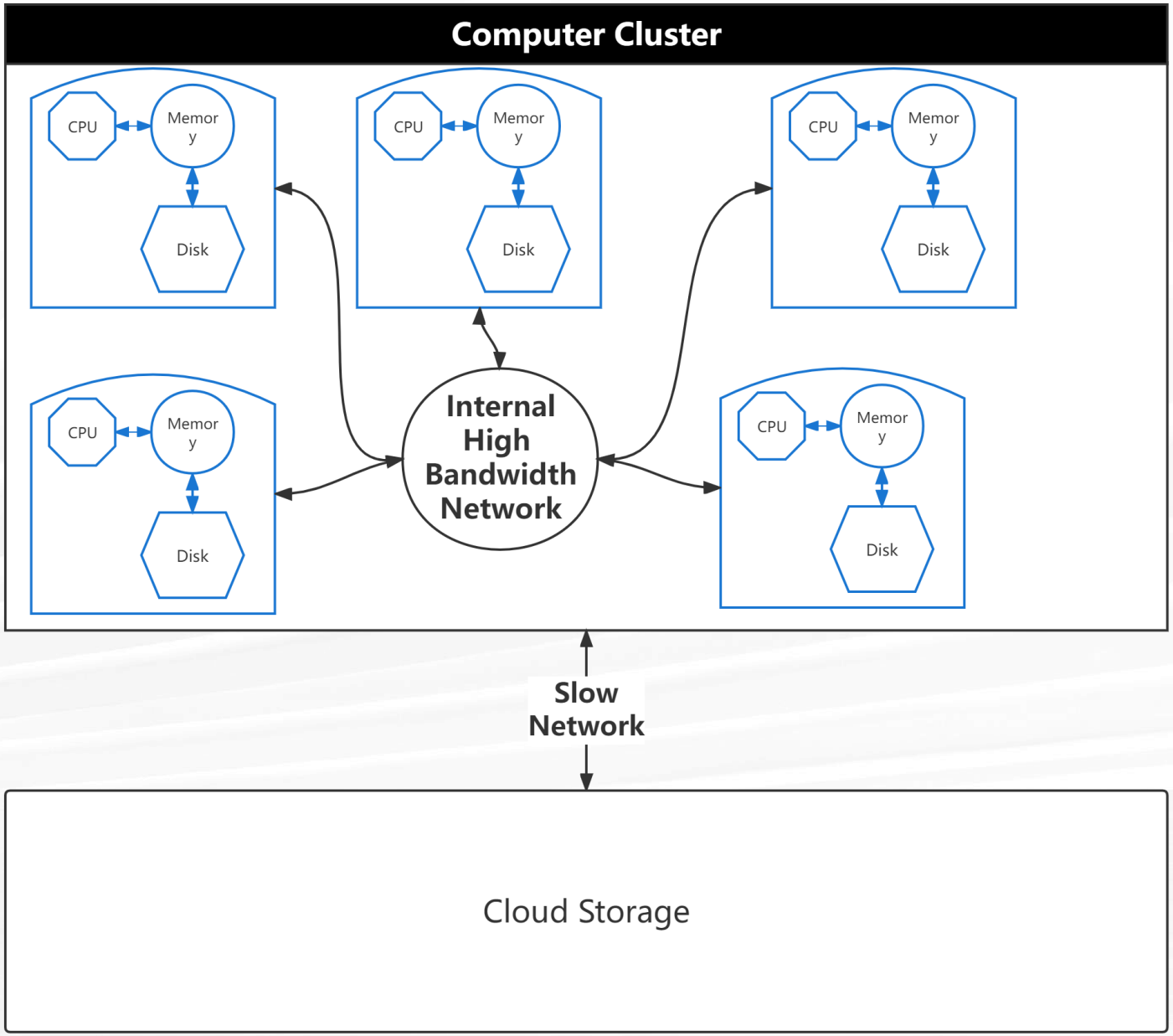
# 多核GEMM

- SUMMA简介
- 性能分析
- 初版设计
- SUMMA Variant

# SUMMA简介

Tecorigin 太初元碁

**Internal network utilization** **!**

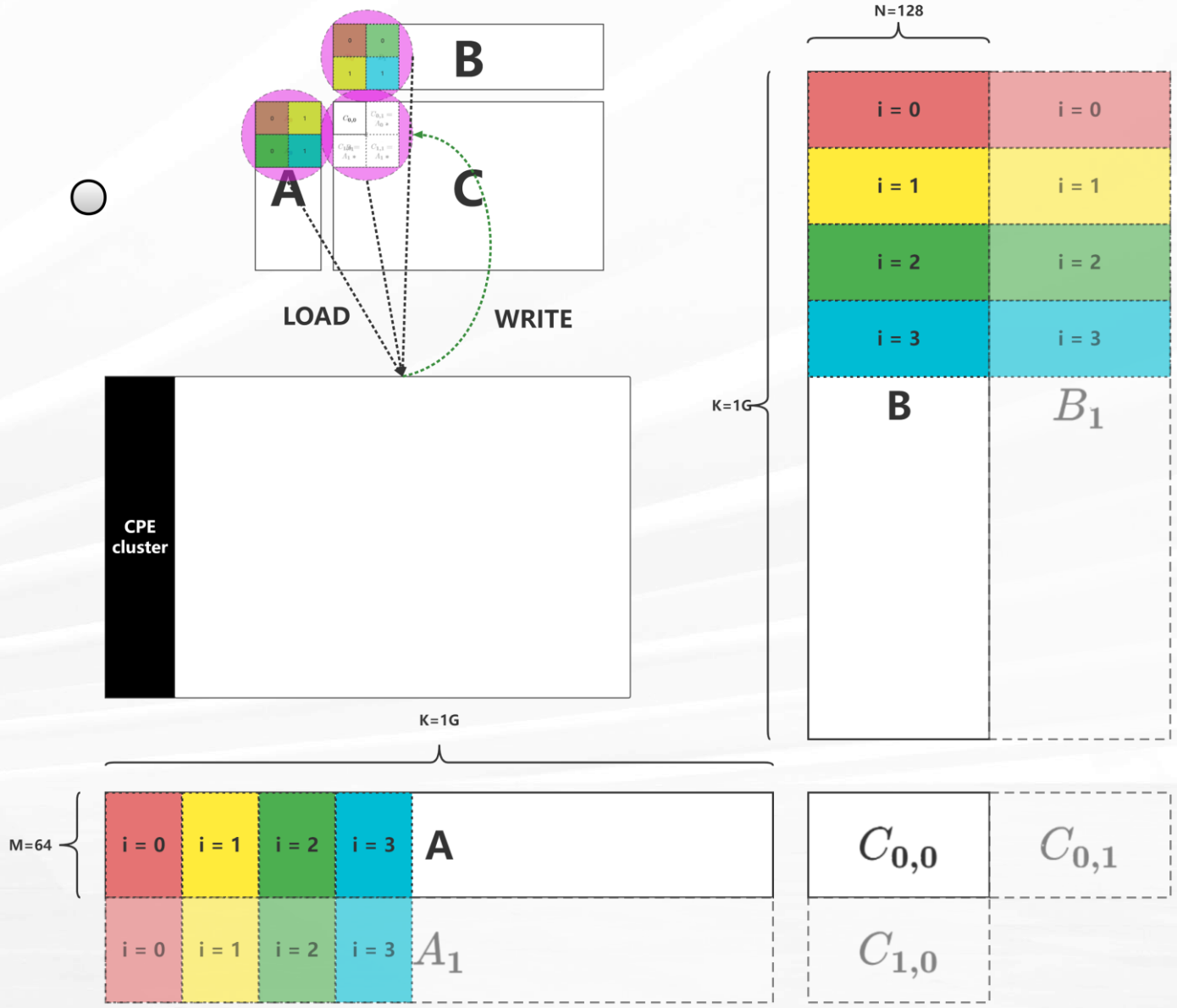



多核 GEMM

# SUMMA简介

Tecorigin 太初元碁

## View CPE cluster as a single CPU

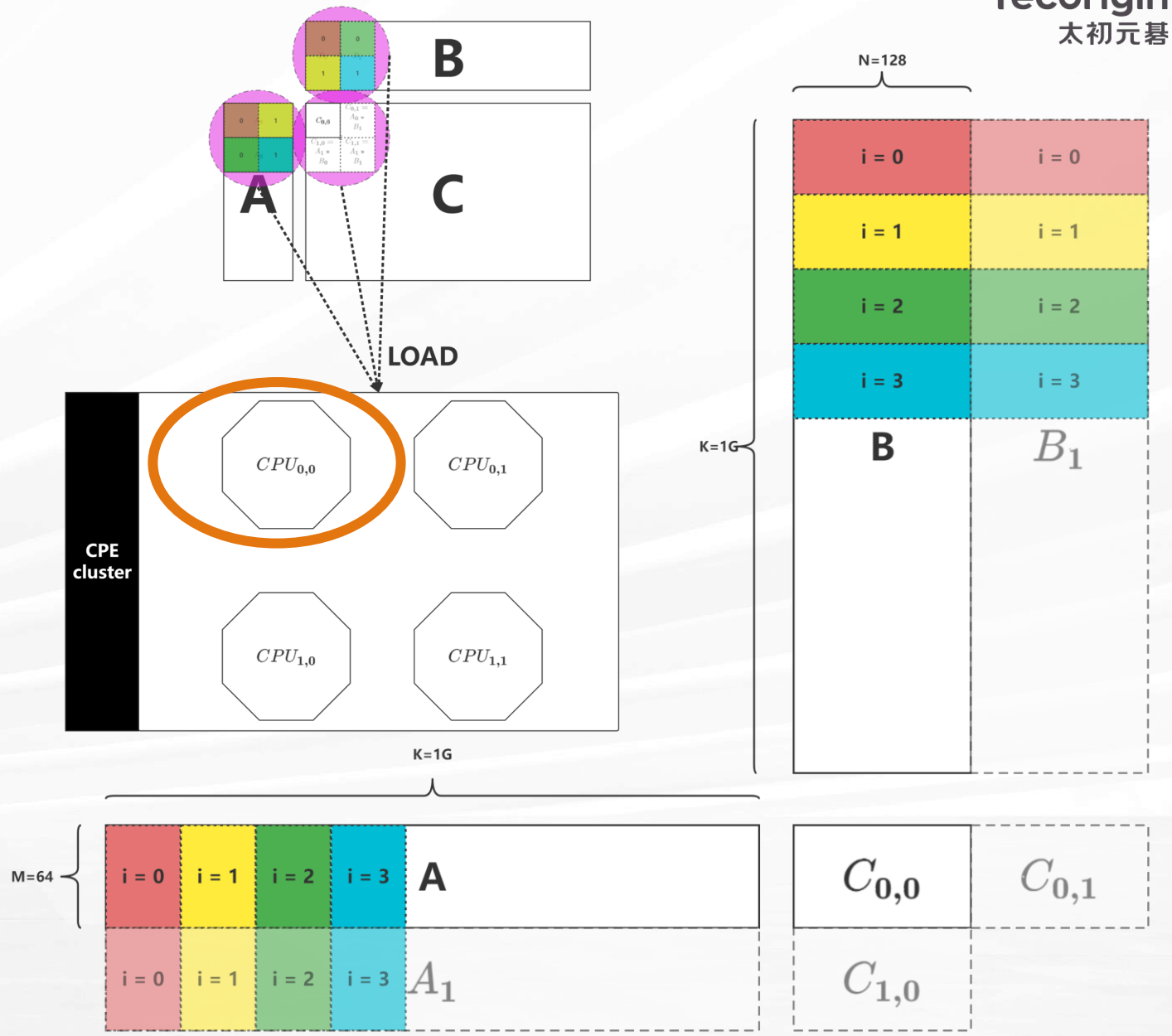



多核 GEMM

# SUMMA简介

Tecorigin 太初元碁

## View CPE cluster as a single CPU

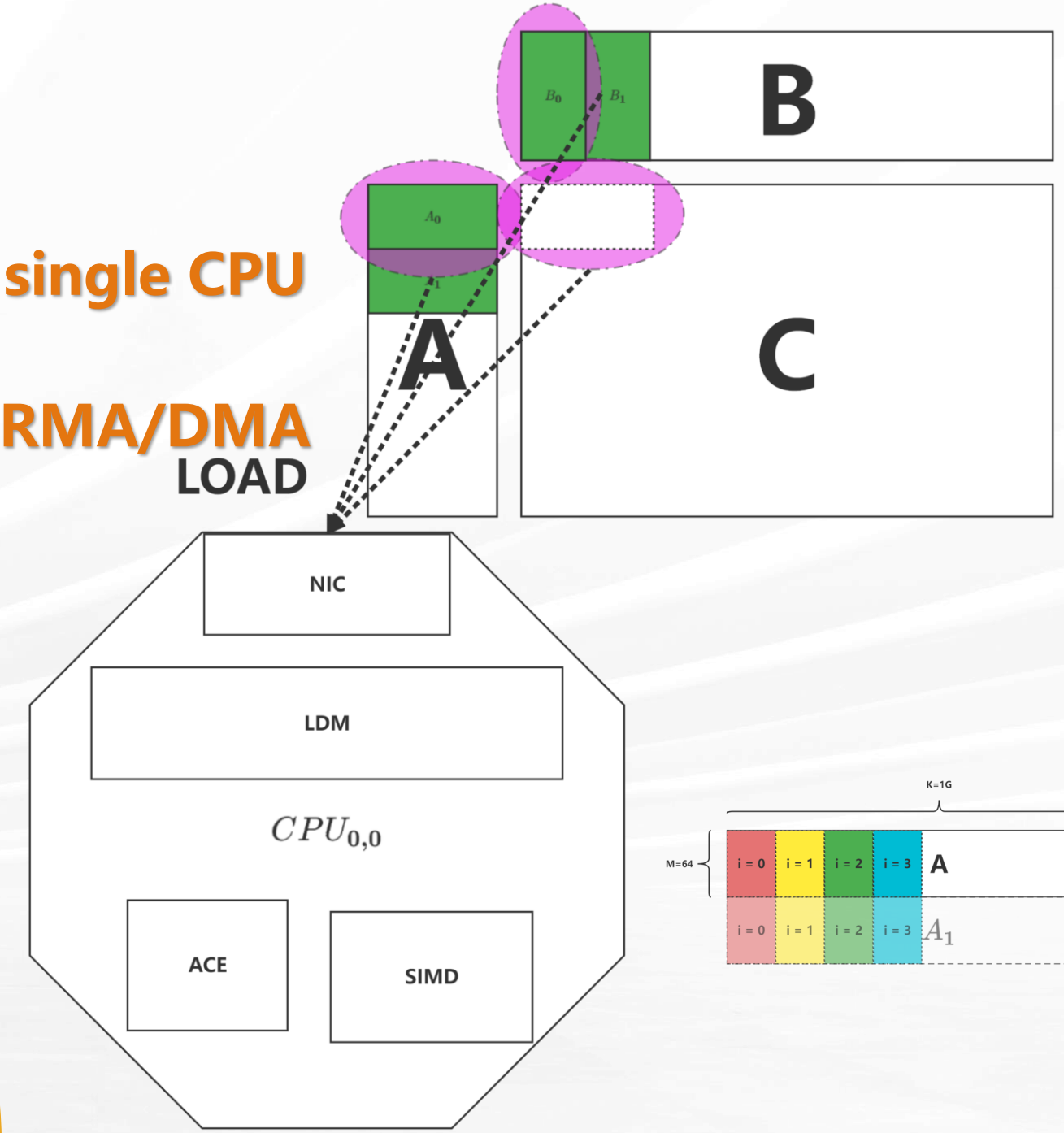



# 多核 GEMM

# SUMMA简介

Tecorigin 太初元碁

## Each core work as single CPU

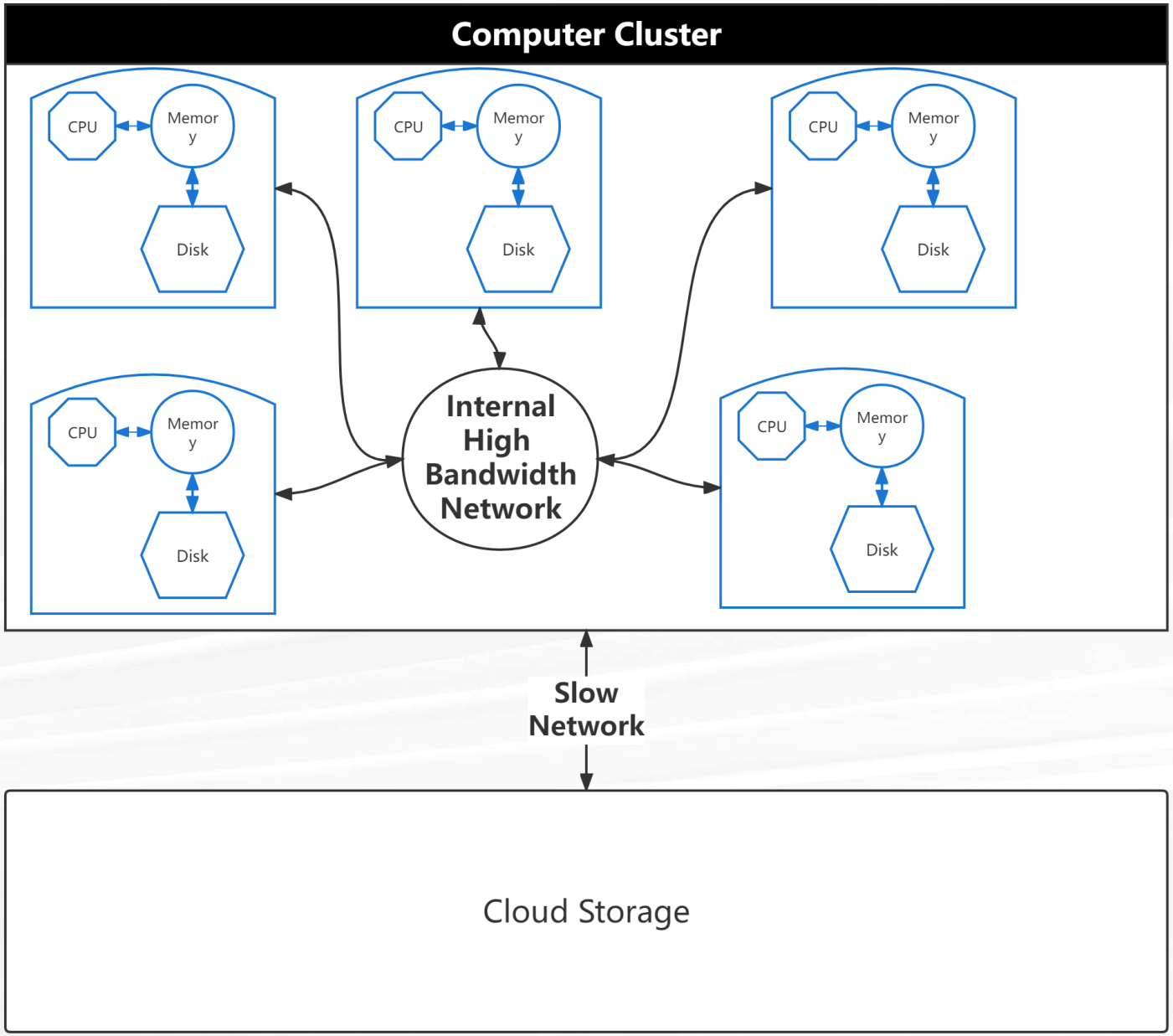



# 多核 GEMM

# SUMMA简介

Tecorigin 太初元碁

**Internal network utilization** **!**

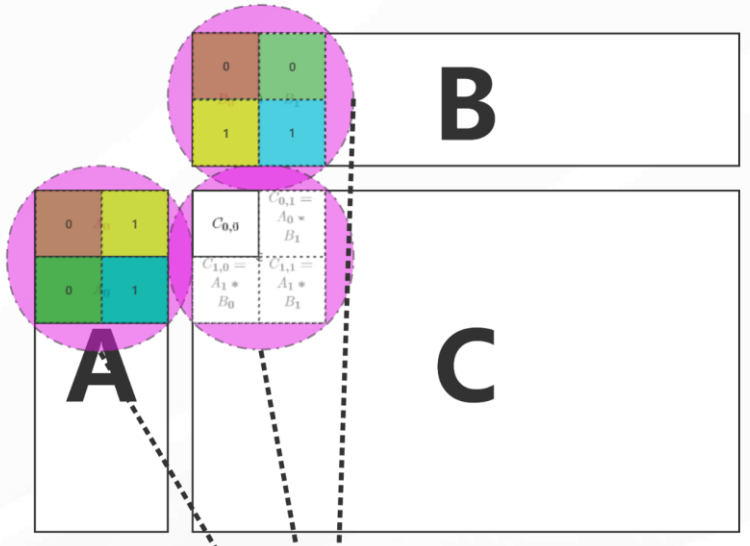


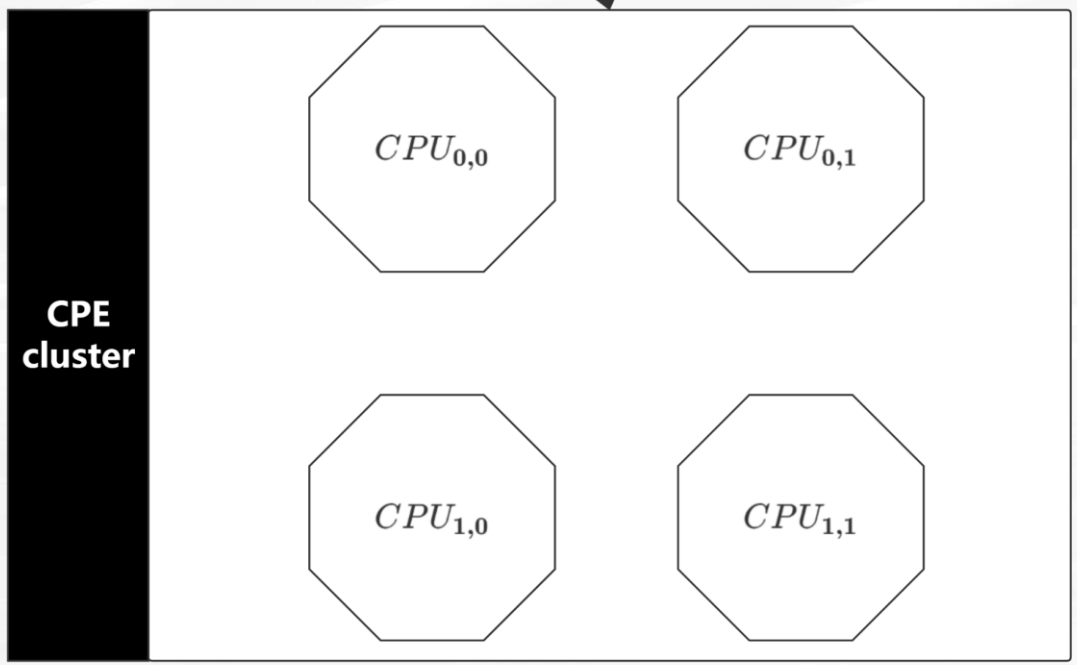

多核 GEMM

# SUMMA简介

Tecorigin
太初元碁

RMA
utilization

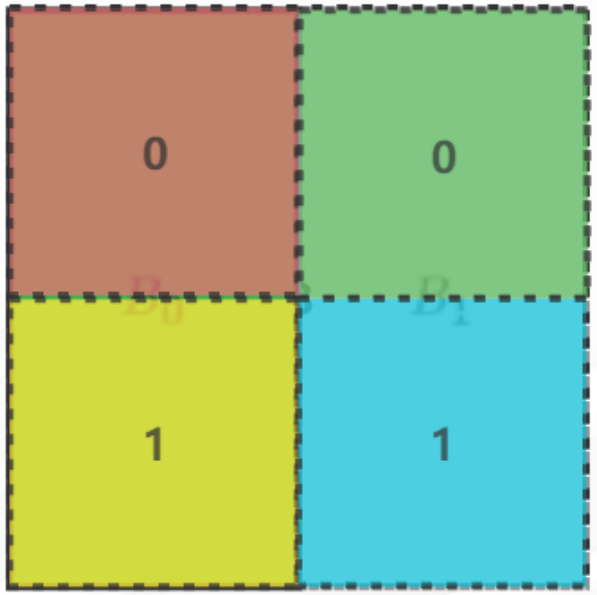


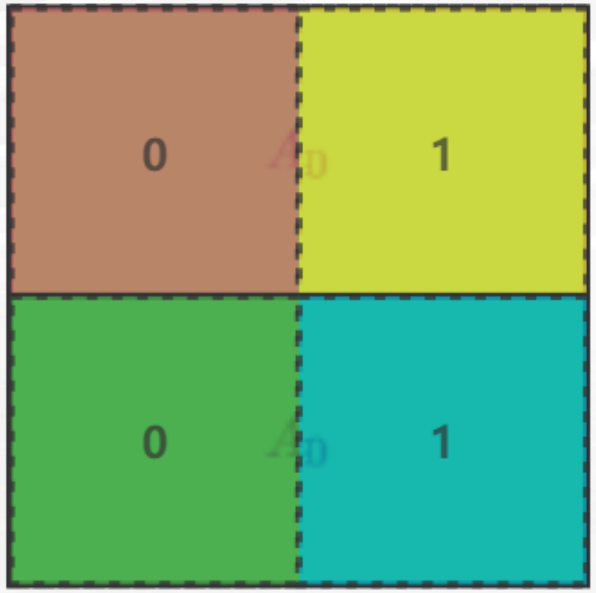



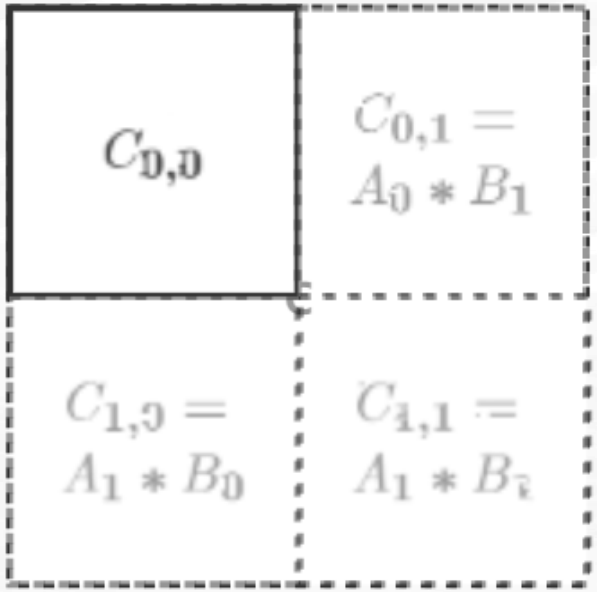

多核
GEMM

Tecorigin 太初元碁

# Example



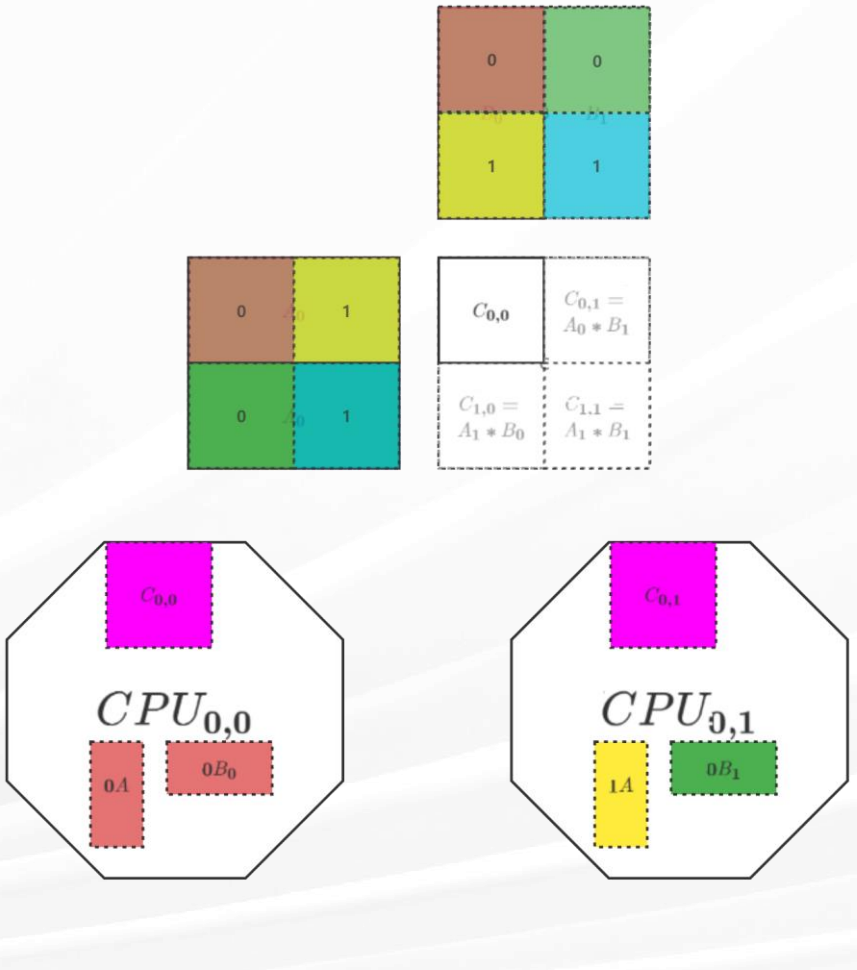




多核 GEMM

# SUMMA简介

Tecorigin
太初元碁

## Load

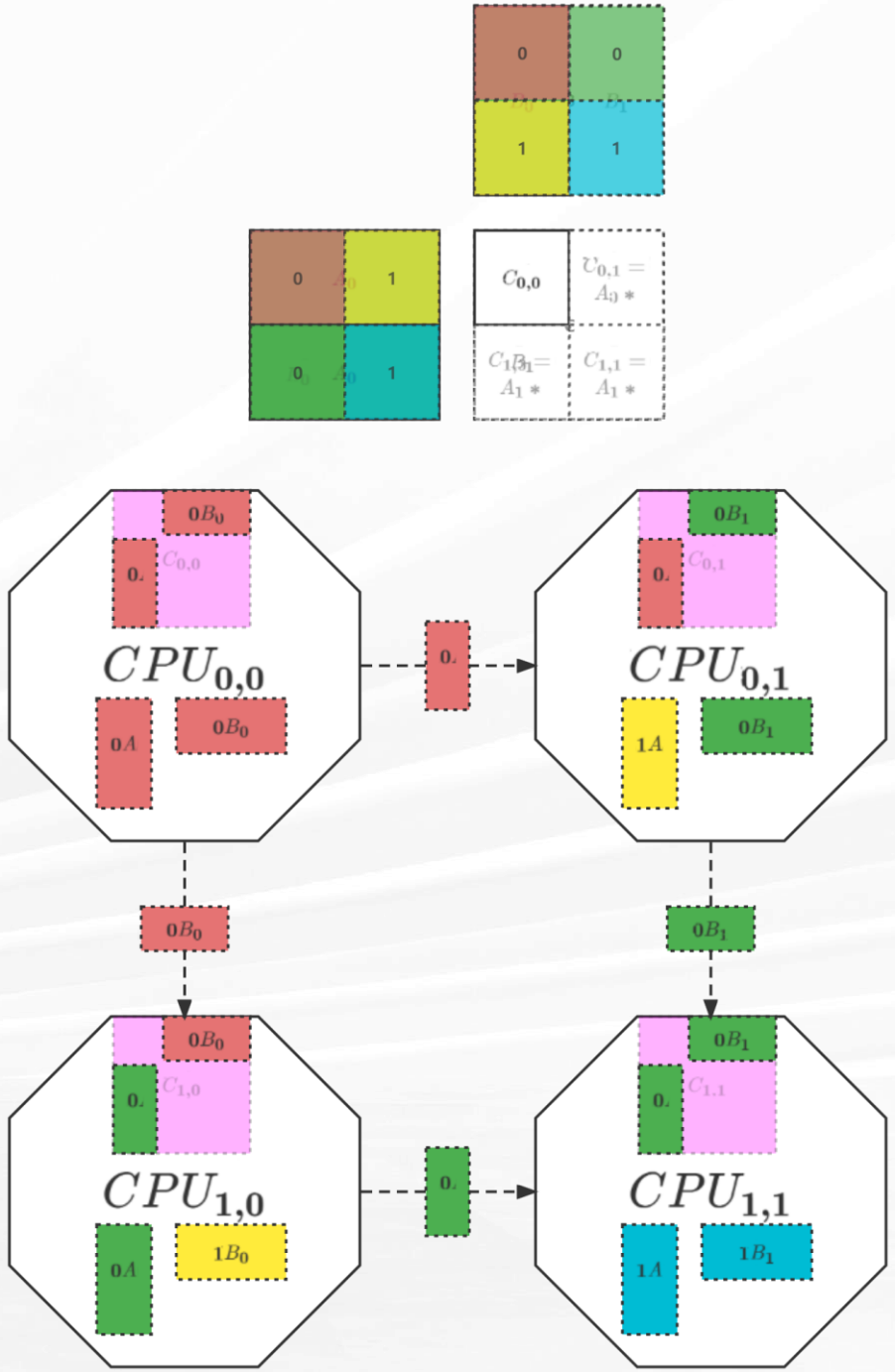



# 多核 GEMM

# SUMMA简介

Tecorigin
太初元碁

## Loop 1

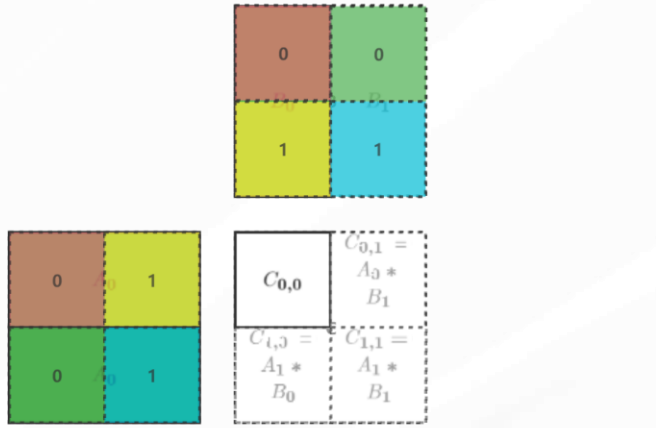


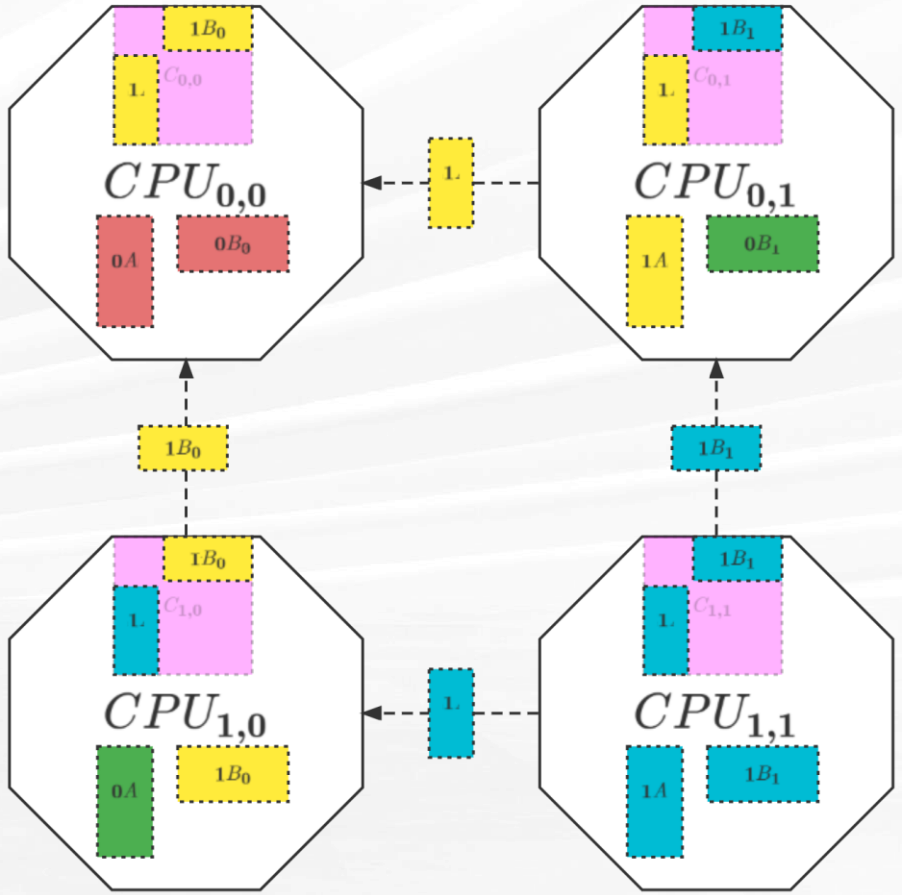

多核
GEMM

# SUMMA简介

Tecorigin
太初元碁

## Loop 2




# 多核 GEMM

# SUMMA简介

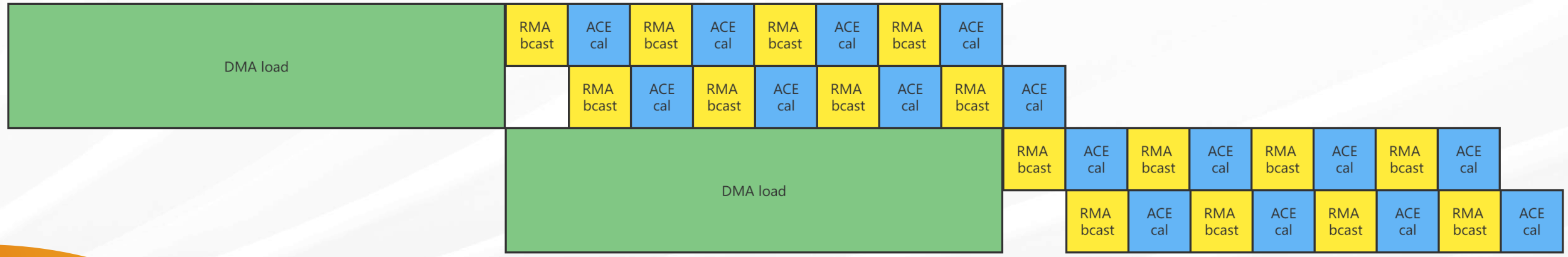
Tecorigin 太初元碁

多核 GEMM

```
for i_L1 in range(0, number_i_L1):
    for j_L1 in range(0, number_j_L1):
        # load C[i_L1,j_L1]
        every core load C[i_L1 + i_L2*rid, j_L1 + j_L2*cid]
        # mark as C_L2
        for p_L1 in range(0, number_k_L1):
            every core load A[i_L1, p_L1 + k_L2_A*cid]
            every core load B[p_L1 + k_L2_B*rid, j_L2]
            for q_L2 in range(0, number_k_L2):
                row_bcast A[i_L1,q_L2] as A_L2
                col_bcast B[q_L2,j_L1] as B_L2
                compute C_L2 += A_L2*B_L2
        store C[i_L1,j_L1]
```

# 性能分析

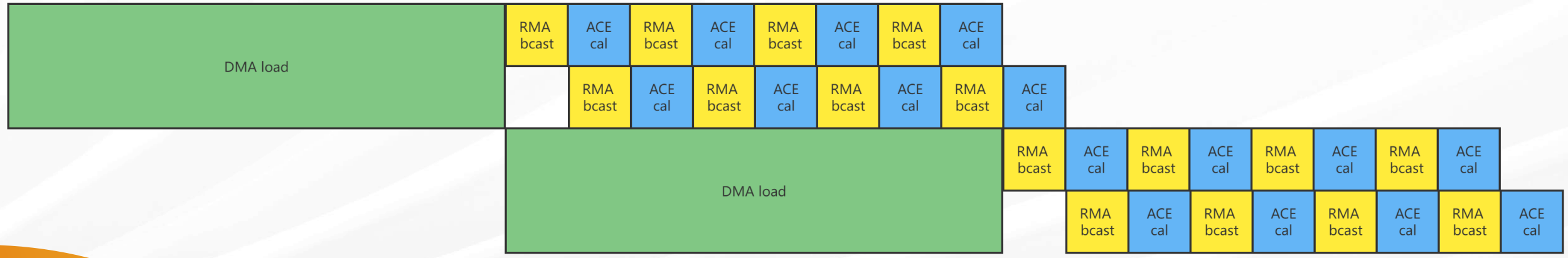
Tecorigin
太初元碁




## 多核 GEMM

# 性能分析

Tecorigin 太初元碁

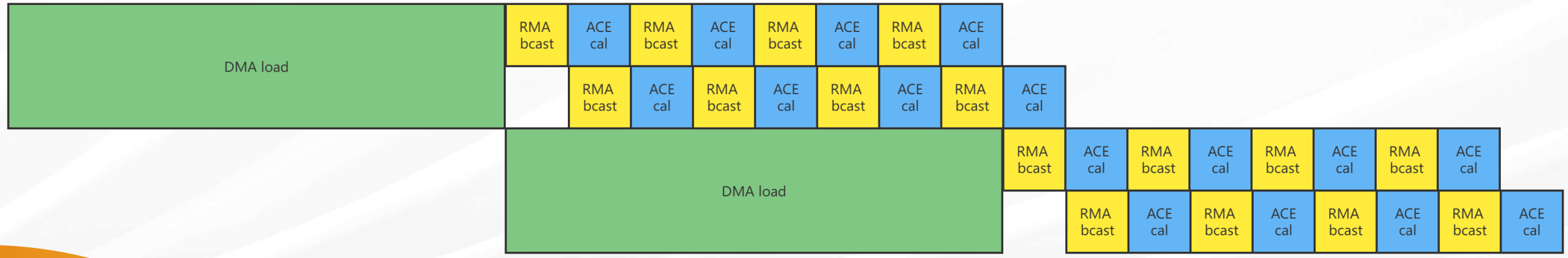



多核 GEMM

$$\begin{cases} T_{dma} < T_{ace} \\ T_{rma} < T_{ace} \end{cases}$$

# 性能分析

Tecorigin 太初元碁



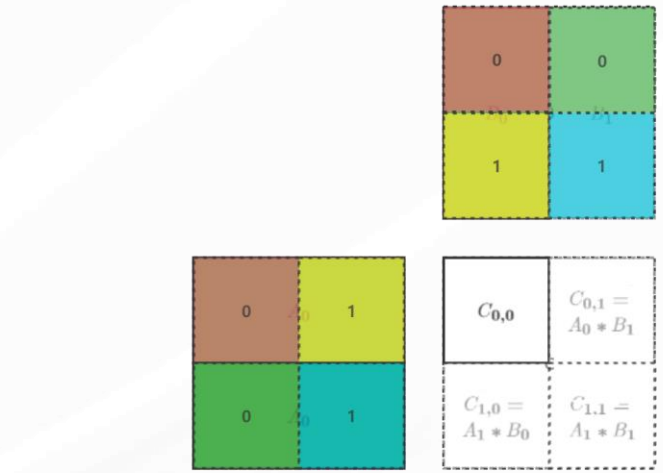

## 多核 GEMM

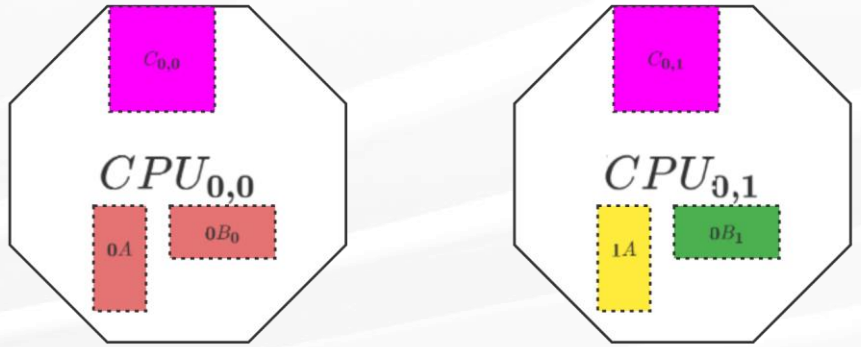
$$\begin{cases} \dfrac{Amount_{ace}}{Amount_{dma}} > \dfrac{Performance_{ace}}{Bandwidth_{dma}} \\ \dfrac{Amount_{ace_cpe}}{Amount_{rma}} > \dfrac{Performance_{ace_cpe}}{Bandwidth_{rma}} \end{cases}$$

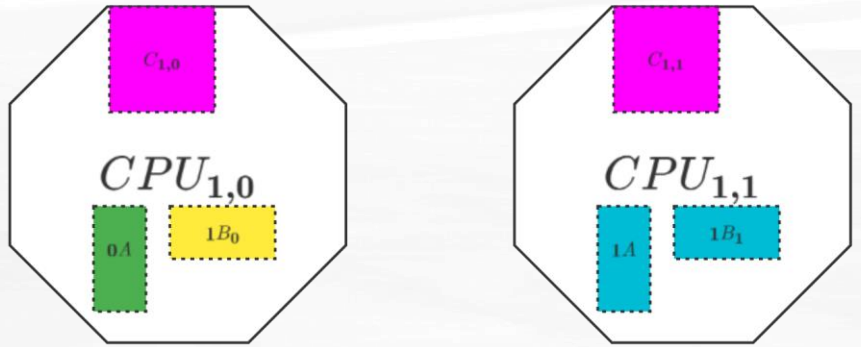
# 性能分析

Tecorigin
太初元碁

$Amount_{dma}$?



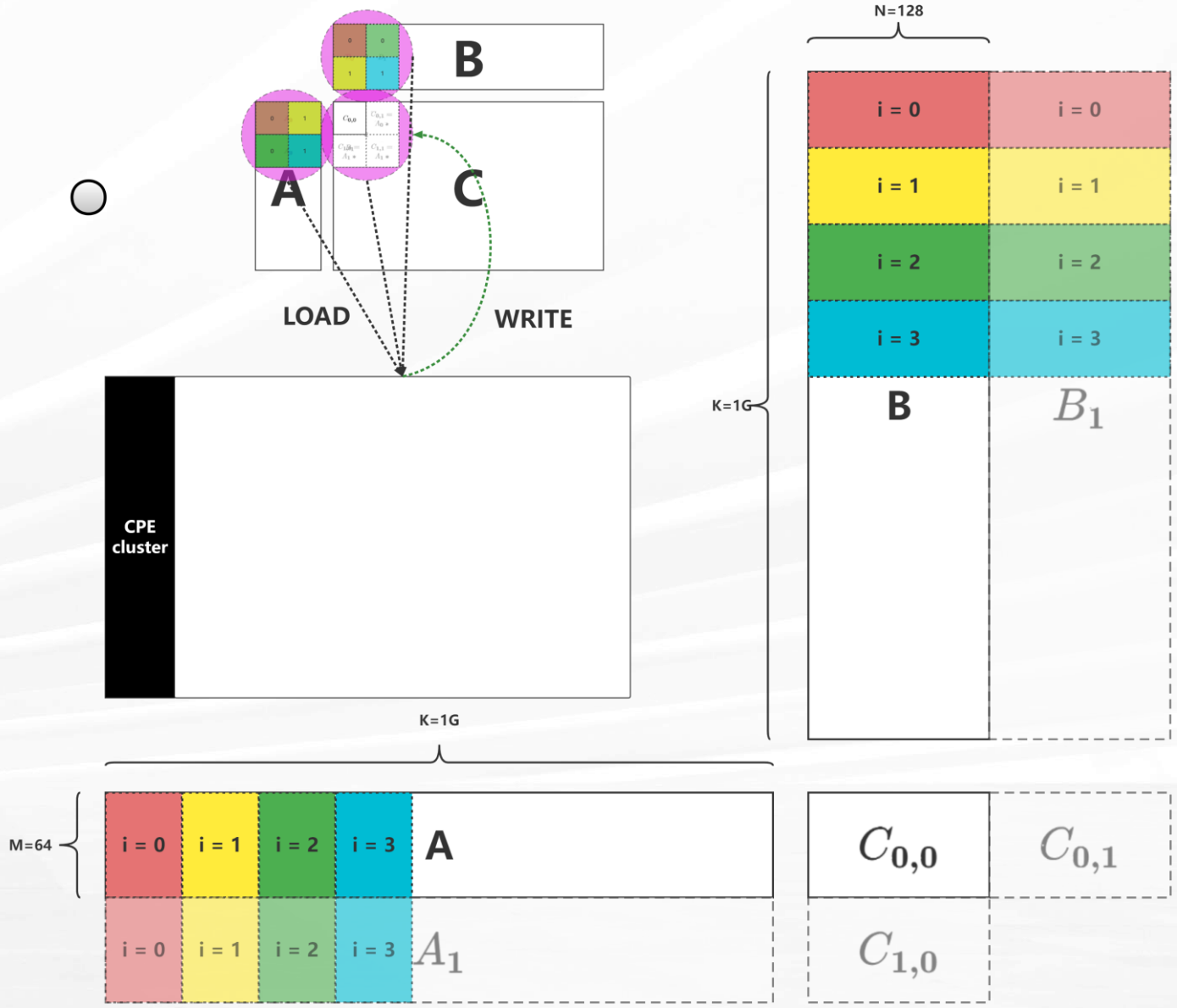




多核
GEMM

# 性能分析

Tecorigin 太初元碁

## View CPE cluster as a single CPU

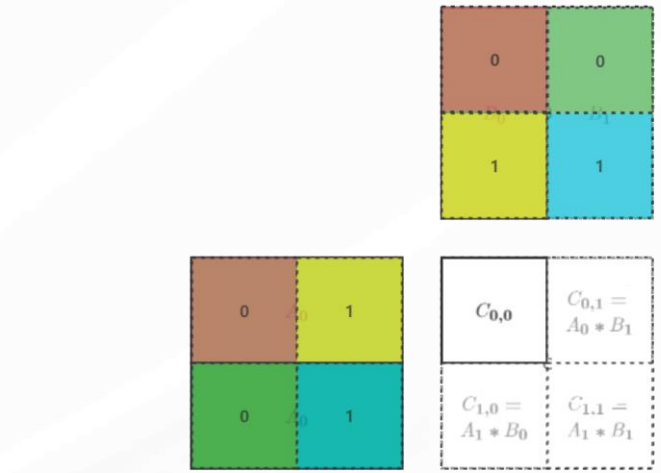


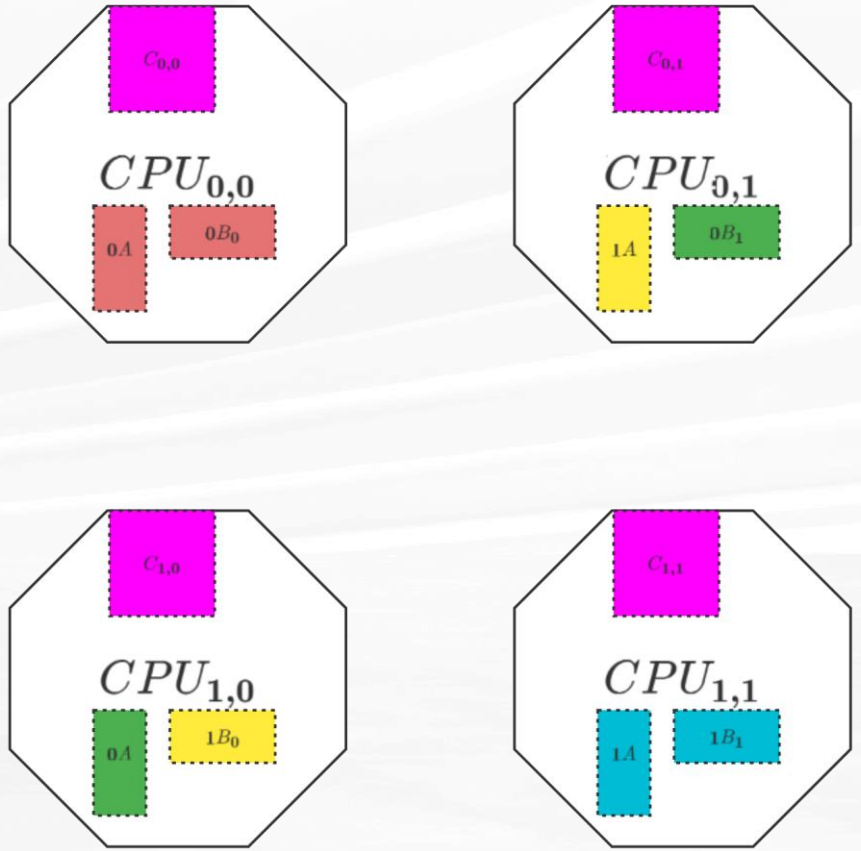

# 多核 GEMM

# 性能分析

Tecorigin 太初元碁

$$Amount_{dma} = 2mnk(\frac{1}{n_{L1}} + \frac{1}{m_{L1}})$$



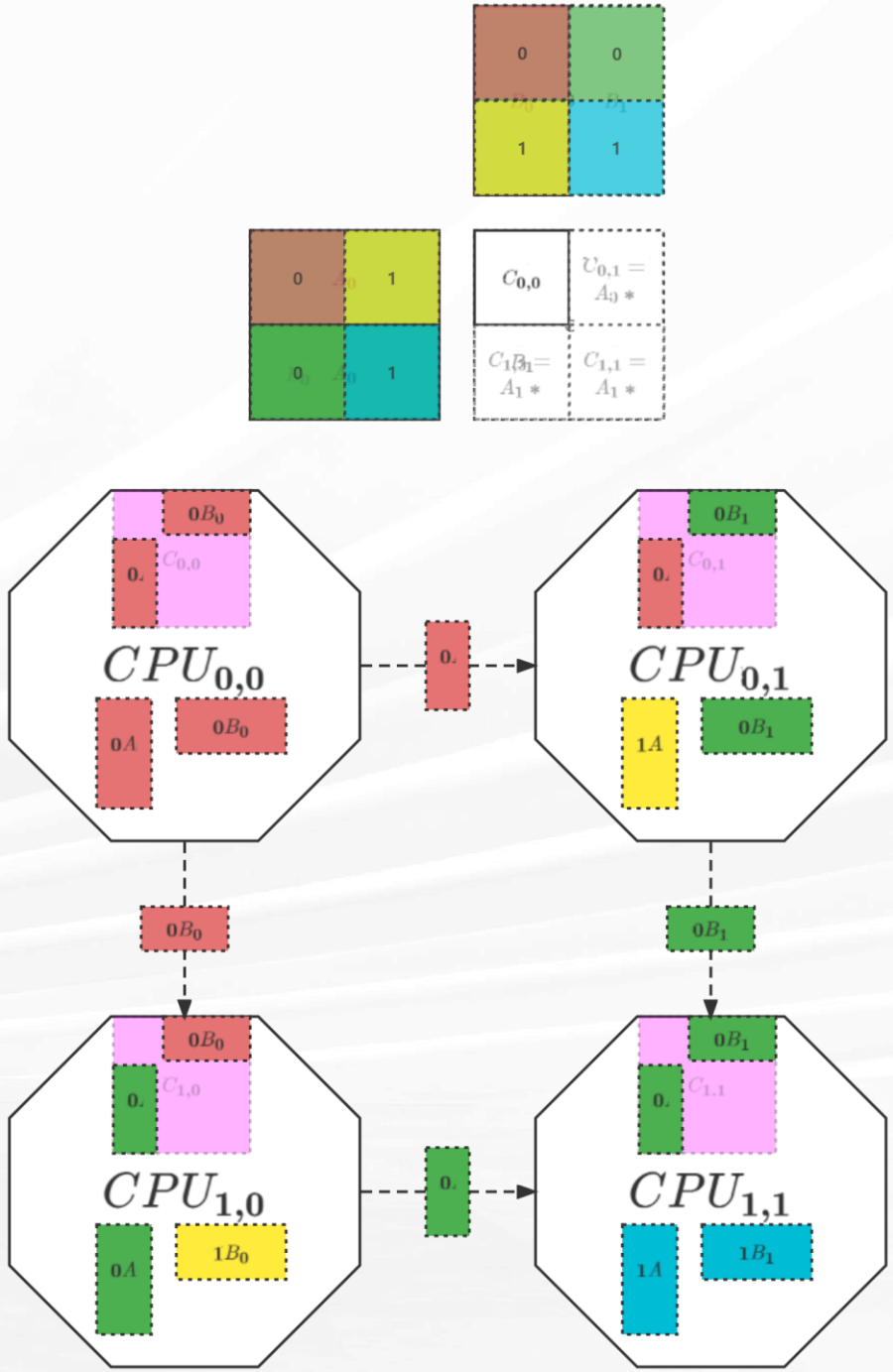




# 多核 GEMM

# 性能分析

Tecorigin
太初元碁

$Amount_{rma}$?

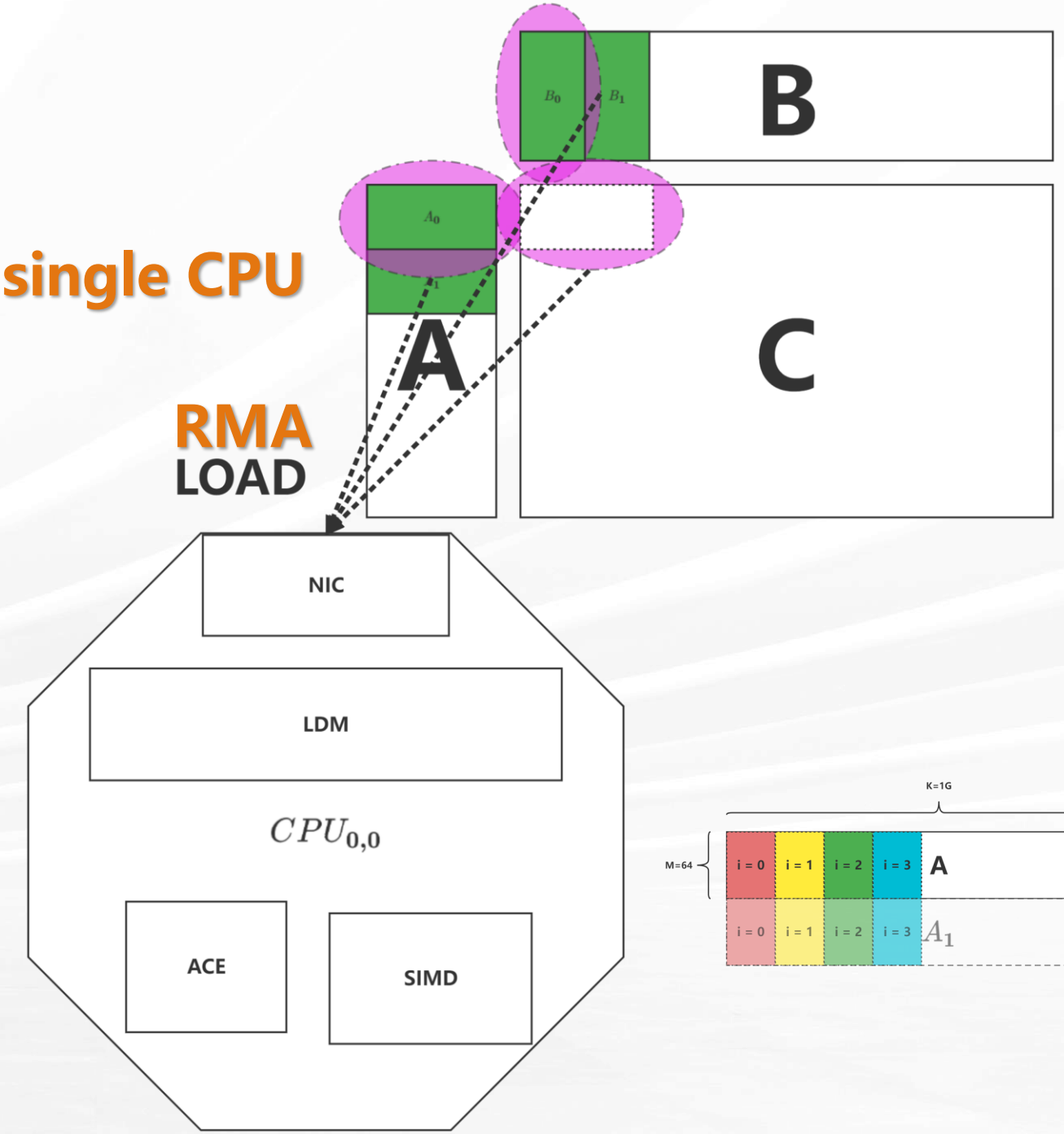



# 多核 GEMM

# 性能分析

Tecorigin 太初元碁

## Each core work as single CPU



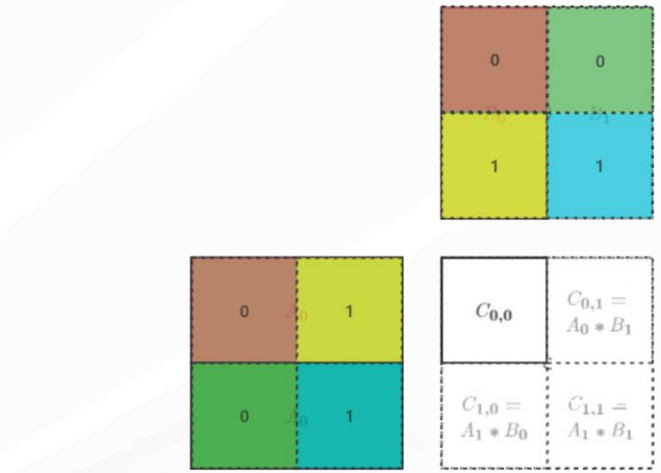

# 多核 GEMM

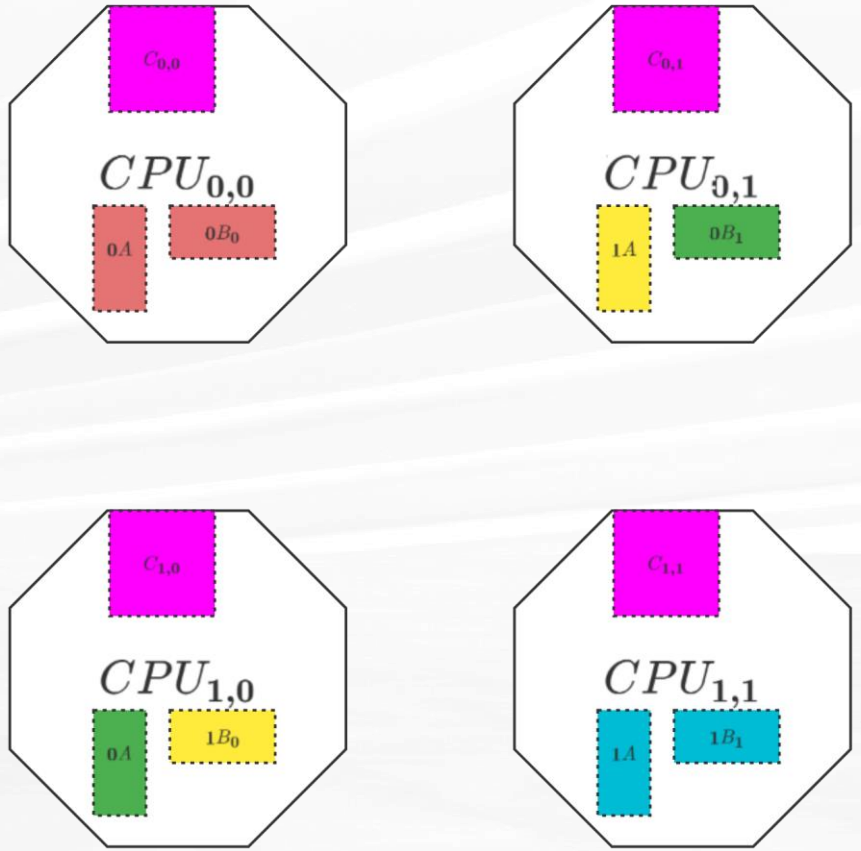
# 性能分析

Tecorigin
太初元碁

## For a single core:

$$Amount_{rma} = 2mnk(\frac{1}{n_{L2}} + \frac{1}{m_{L2}})$$



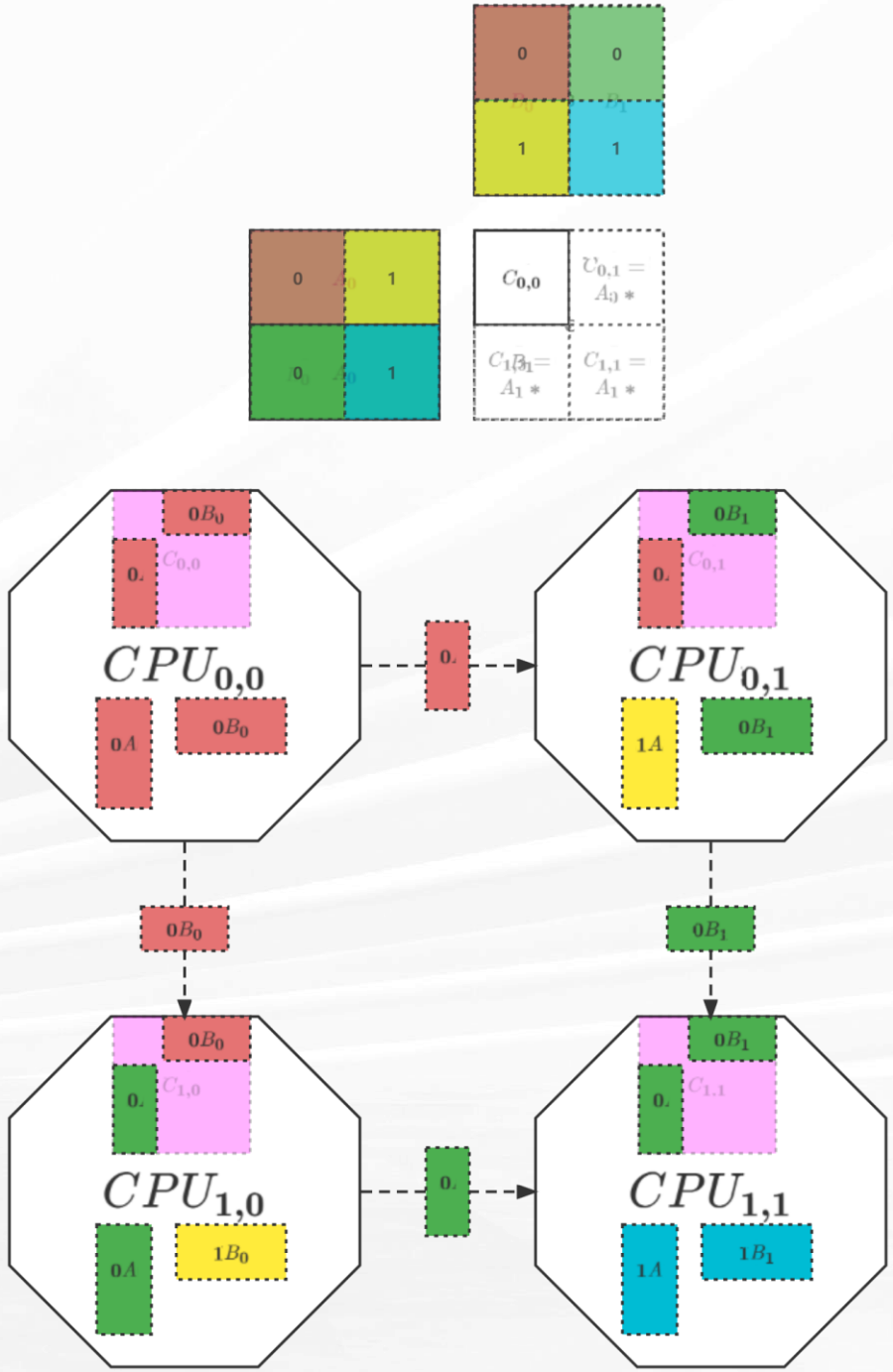




# 多核 GEMM

# 性能分析

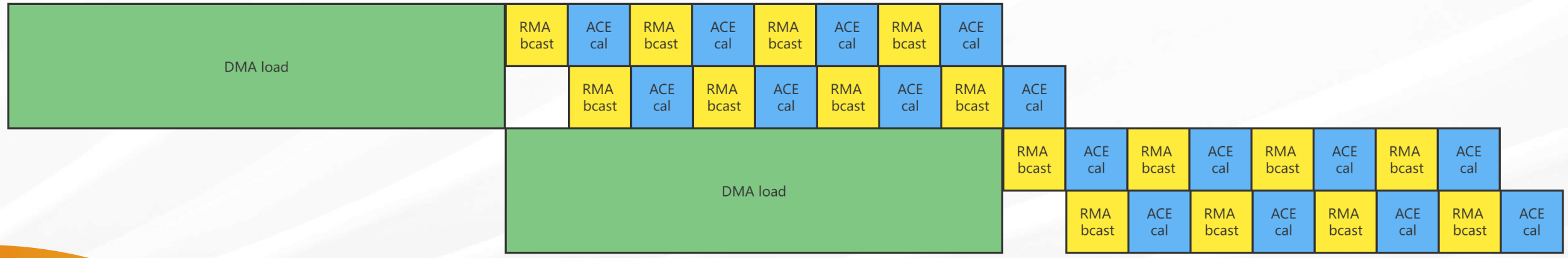
Tecorigin
太初元碁

$$Amount_{ace} = 2mnk$$




多核
GEMM

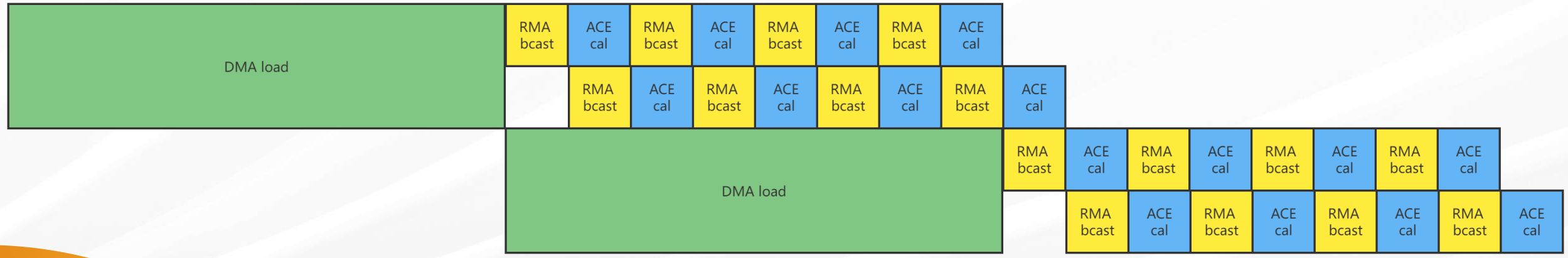
# 性能分析




# 多核 GEMM

$$\begin{cases} \dfrac{2mnk}{2mnk(\frac{1}{n_{L1}}+\frac{1}{m_{L1}})} > \dfrac{\mathrm{Per}formance_{ace}}{\mathrm{Bandwidth}_{dma}} \\ \dfrac{2mnk}{2mnk(\frac{1}{n_{L2}}+\frac{1}{m_{L2}})} > \dfrac{\mathrm{Per}formance_{ace_cpe}}{\mathrm{Bandwidth}_{rma}} \end{cases}$$

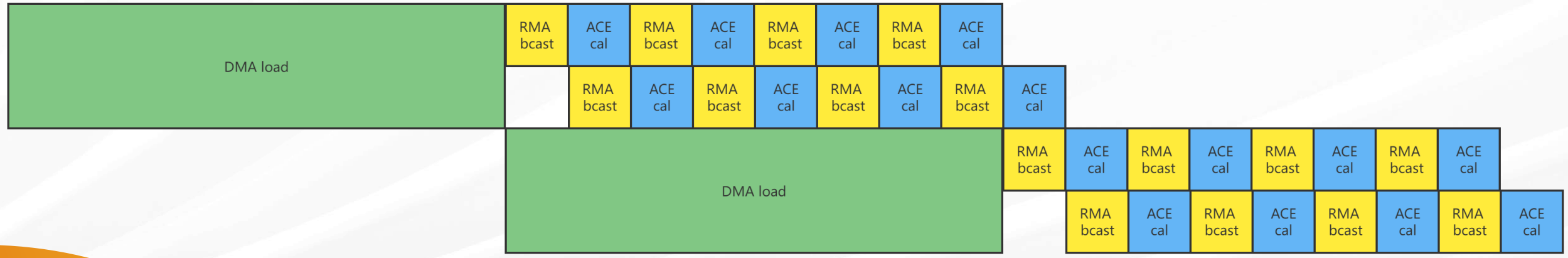
# 性能分析




# 多核 GEMM

$$\begin{cases} \dfrac{n_{L1}m_{L1}}{n_{L1}+m_{L1}} > \dfrac{\mathrm{Per}formance_{ace}}{\mathrm{Bandwidth}_{dma}} \\ \dfrac{n_{L2}m_{L2}}{n_{L2}+m_{L2}} > \dfrac{\mathrm{Per}formance_{ace_cpe}}{\mathrm{Bandwidth}_{rma}} \end{cases}$$

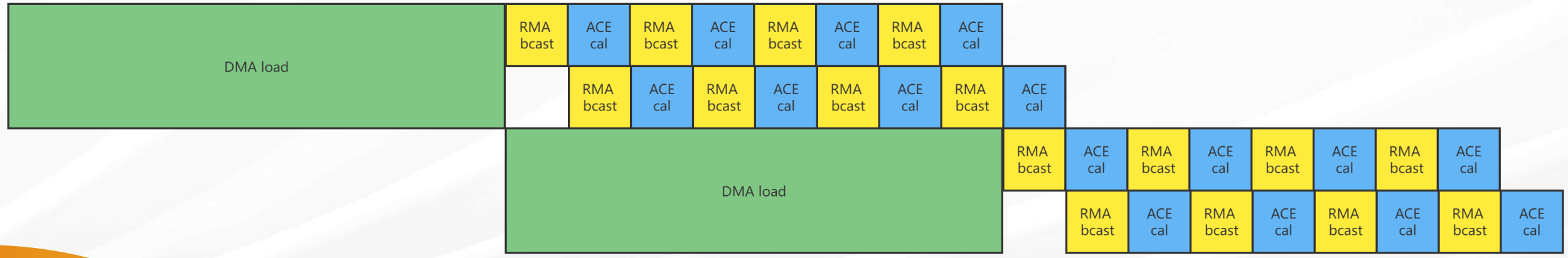
# 性能分析




多核 GEMM

$$\begin{cases} \dfrac{n_{L1}m_{L1}}{n_{L1}+m_{L1}} > \dfrac{80000\ Gflops}{250GB/s} \\ \dfrac{n_{L2}m_{L2}}{n_{L2}+m_{L2}} > \dfrac{2560\ Gflops}{64GB/s} \end{cases}$$

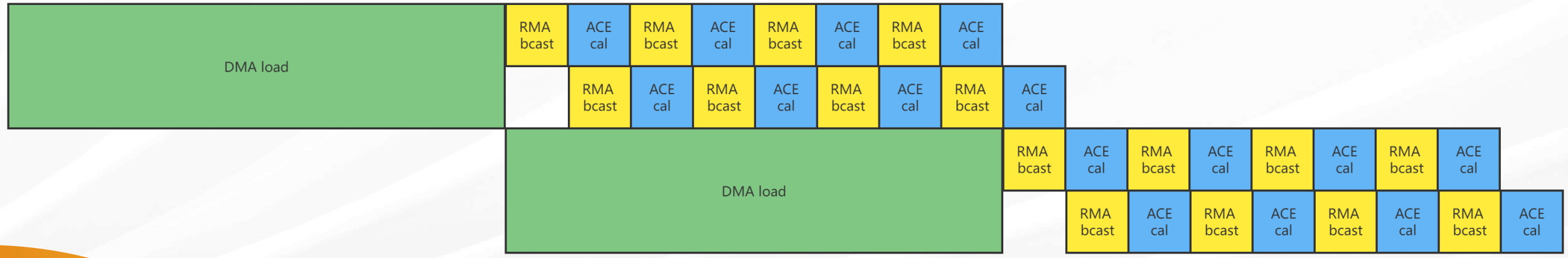
# 性能分析




# 多核 GEMM

$$\begin{cases} \dfrac{n_{L1}m_{L1}}{n_{L1}+m_{L1}} > 320 \\ \dfrac{n_{L2}m_{L2}}{n_{L2}+m_{L2}} > 40 \end{cases}$$

# 性能分析



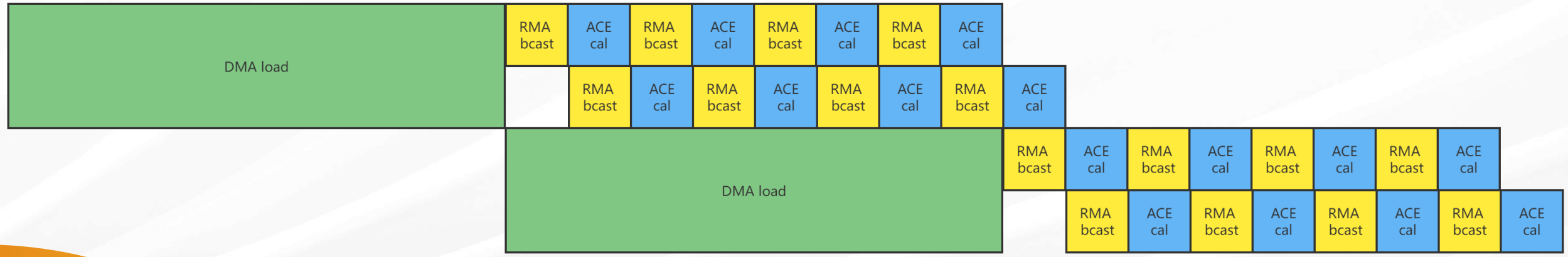

## 多核 GEMM

$$\begin{cases} n_{L1} = m_{L1} > 640 \\ n_{L2} = m_{L2} > 80 \end{cases}$$

$$\begin{cases} n_{L1} = 8n_{L2} \\ m_{L1} = 4m_{L2} \end{cases}$$

# 初版设计

Tecorigin 太初元碁

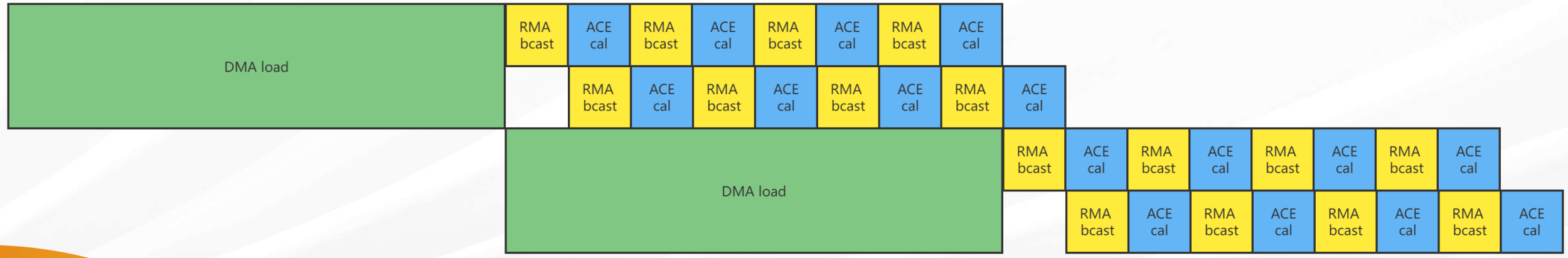



# 多核 GEMM

$$\begin{cases} \dfrac{n_{L1}m_{L1}}{n_{L1}+m_{L1}} > 320 \\ \dfrac{n_{L2}m_{L2}}{n_{L2}+m_{L2}} > 40 \end{cases}$$

$$\begin{cases} n_{L1} = 8n_{L2} \\ m_{L1} = 4m_{L2} \end{cases}$$

# 初版设计




$$\begin{cases} \dfrac{n_{L1}m_{L1}}{n_{L1}+m_{L1}} > 320 \\ \dfrac{n_{L2}m_{L2}}{n_{L2}+m_{L2}} > 40 \end{cases}$$

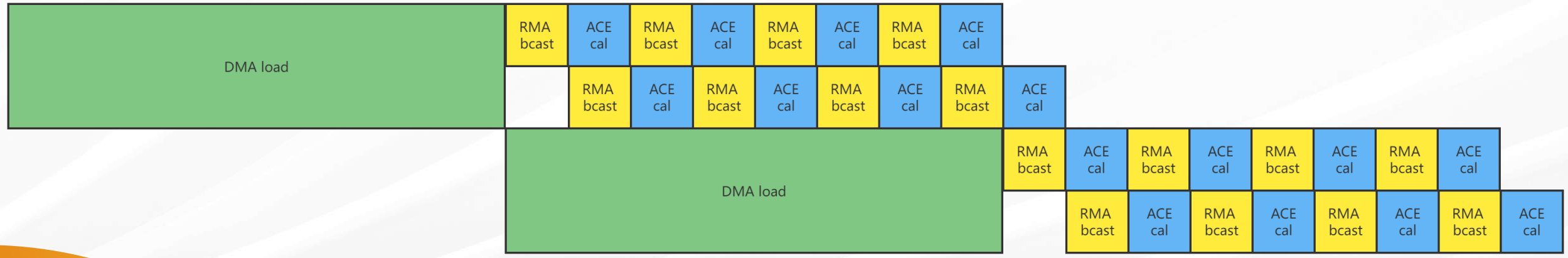
$$\begin{cases} n_{L1} = 8n_{L2} \\ m_{L1} = 4m_{L2} \end{cases}$$

# 多核 GEMM

$$\begin{cases} n_{L2} = 128 \\ m_{L2} = 128 \end{cases}$$

$$\begin{cases} n_{L1} = 8 * 128 = 1024 \\ m_{L1} = 4 * 128 = 512 \end{cases}$$

# 初版设计




$$\begin{cases} \dfrac{n_{L1}m_{L1}}{n_{L1}+m_{L1}} > 320 \\ \dfrac{n_{L2}m_{L2}}{n_{L2}+m_{L2}} > 40 \end{cases} \surd$$

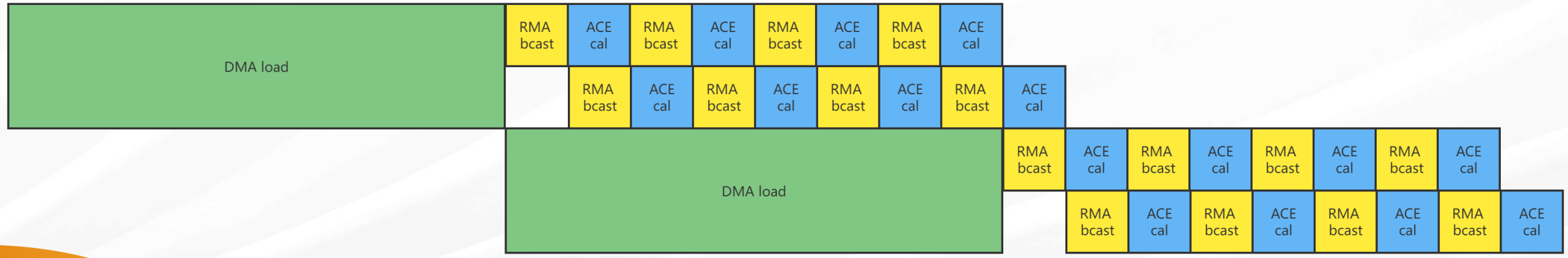
$$\begin{cases} n_{L1} = 8n_{L2} \\ m_{L1} = 4m_{L2} \end{cases}$$

# 多核 GEMM

$$\begin{cases} n_{L2} = 128 \\ m_{L2} = 128 \end{cases}$$

$$\begin{cases} n_{L1} = 8 * 128 = 1024 \\ m_{L1} = 4 * 128 = 512 \end{cases}$$

# 初版设计

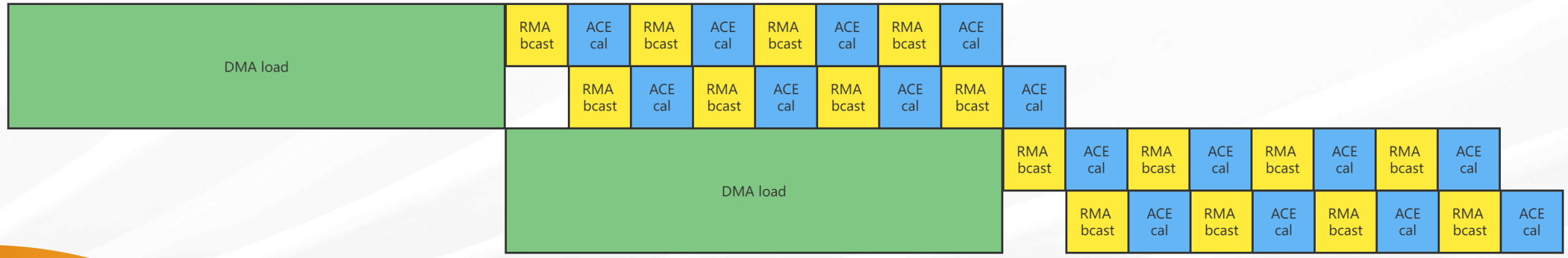



## LDM?

# 多核 GEMM

$$\begin{cases} n_{L2} = 128 \\ m_{L2} = 128 \end{cases} \qquad \begin{cases} n_{L1} = 8 * 128 = 1024 \\ m_{L1} = 4 * 128 = 512 \end{cases}$$

# 初版设计

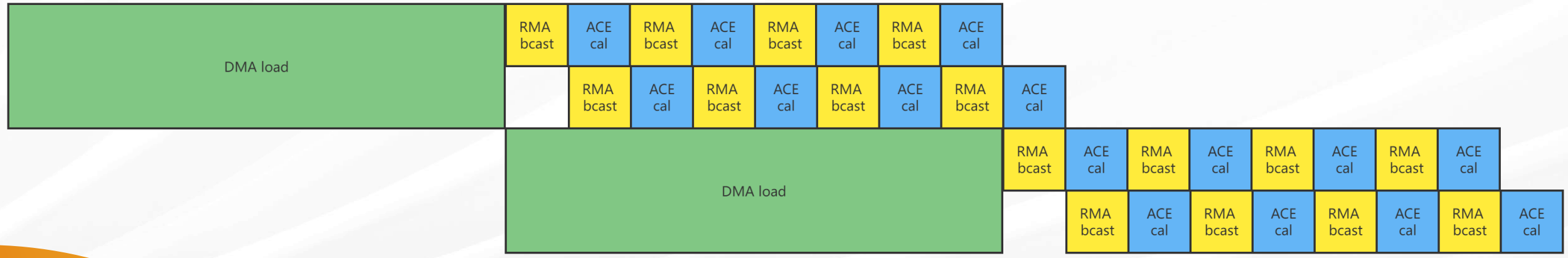



# 多核 GEMM

$$Cost_{ldm_dma} = (Num_A + Num_B) * S_{half} * buf_{double} + Num_C * S_{float} = 4(m_{L2} * k_{L2} + 2n_{L2} * k_{L2} + m_{L2} * n_{L2})$$

$$\begin{cases} n_{L2} = 128 \\ m_{L2} = 128 \end{cases} \qquad \begin{cases} n_{L1} = 8 * 128 = 1024 \\ m_{L1} = 4 * 128 = 512 \end{cases}$$

# 初版设计




多核GEMM

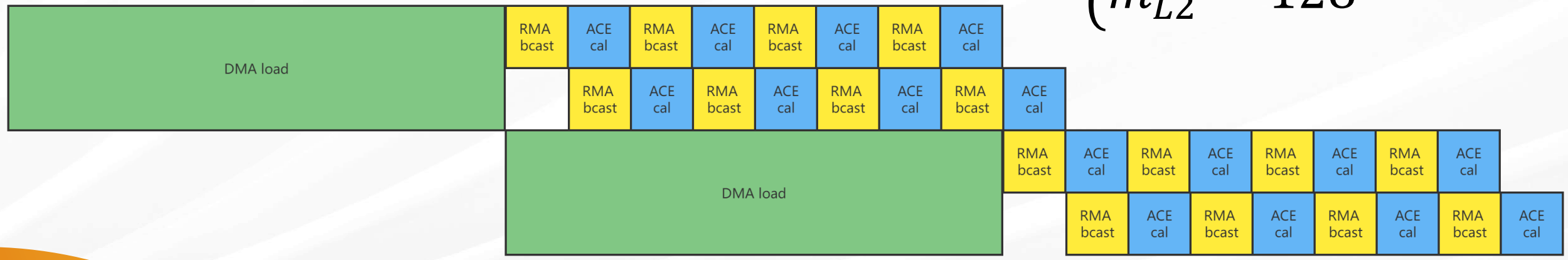
$$Cost_{ldm_rma} = \left(Num_{A_rma} + Num_{B_rma}\right) * S_{half} * buf_{double} = 4(m_{L2} * k_{L2} + k_{L2} * n_{L2})$$

$$\begin{cases} n_{L2} = 128 \\ m_{L2} = 128 \end{cases} \qquad \begin{cases} n_{L1} = 8 * 128 = 1024 \\ m_{L1} = 4 * 128 = 512 \end{cases}$$

# 初版设计

Tecorigin 太初元碁

$$\begin{cases} n_{L2} = 128 \\ m_{L2} = 128 \end{cases}$$



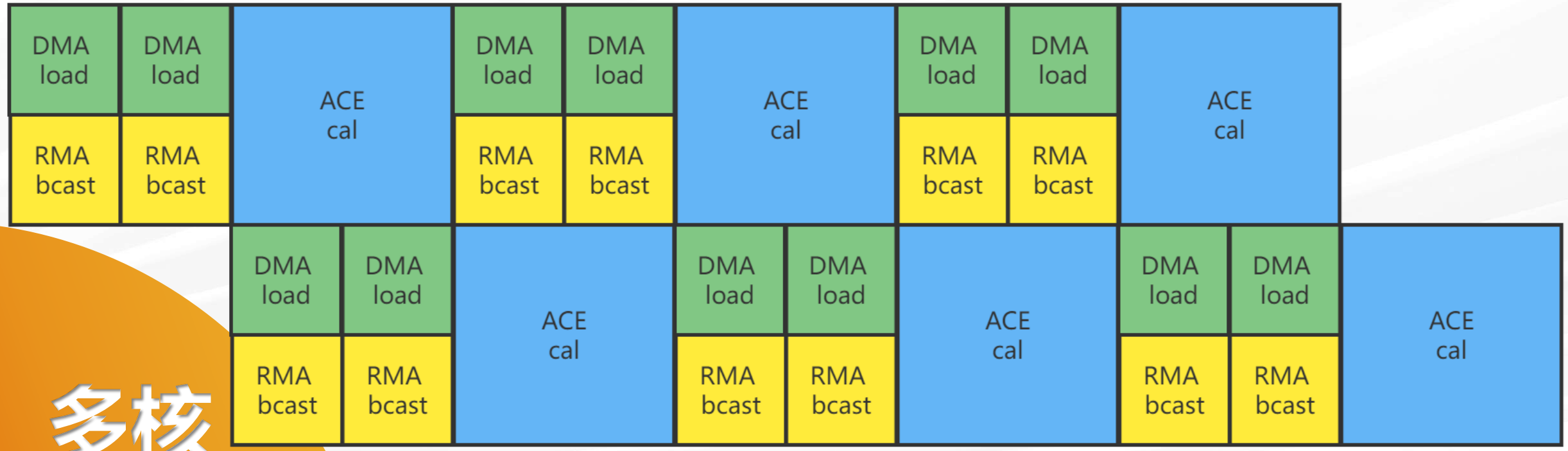

多核 GEMM

$$\begin{aligned} & Cost_{ldm} \\ & = Cost_{ldm_dma} + Cost_{ldm_rma} \\ & = \left(Num_{A_rma} + Num_{B_rma}\right) * S_{half} * buf_{double} \\ & = 4(m_{L2} * k_{L2} + 2k_{L2} * n_{L2}) + 4m_{L2} * n_{L2} + 4(m_{L2} * k_{L2} + k_{L2} * n_{L2}) \\ & = 196KB \end{aligned}$$

$$k_{L2} = 64$$

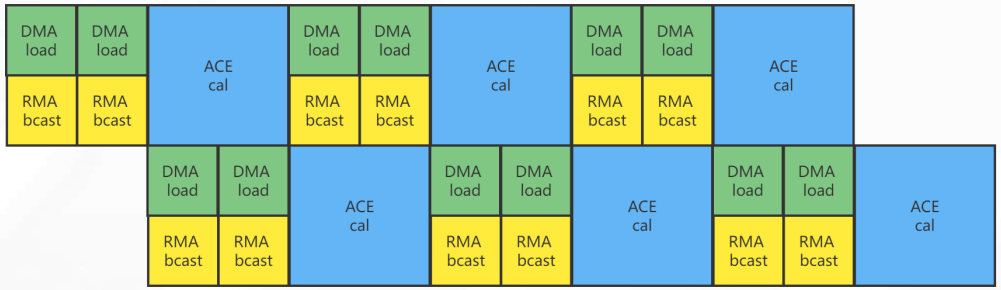
# SUMMA Variant

Tecorigin 太初元碁




多核 GEMM

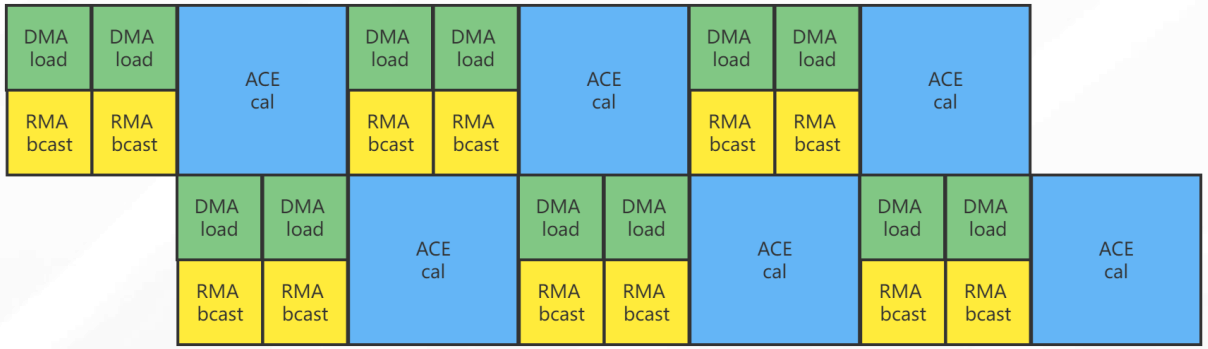
# SUMMA Variant




多核 GEMM

```
for i_L1 in range(0, number_i_L1):
    for j_L1 in range(0, number_j_L1):
        # load C[i_L1,j_L1]
        every core load C[i_L1 + m_L2*rid, j_L1 + n_L2*cid]
        # mark as C_L2
        for p_L1 in range(0, number_k_L1):
            for q_L2 in range(0, number_k_L2):
                dma_row_bcast A[i_L1,p_L1 + q_L2] as A_L2
                dma_col_bcast B[p_L1 + q_L2,j_L1] as B_L2
                compute C_L2 += A_L2*B_L2
        store C[i_L1,j_L1]
```

# SUMMA Variant

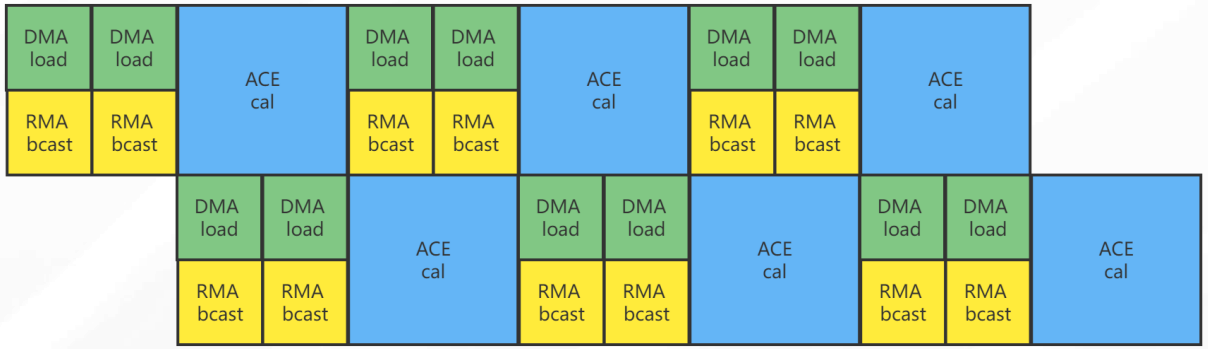



## 满足以下条件时等价与summa

多核 GEMM

$$\begin{cases} T_{dma} < T_{ace} \text{ and } T_{rma} < T_{ace} \\ or \\ T_{dma} < T_{rma} \end{cases}$$

# SUMMA Variant




## 满足以下条件时等价与summa

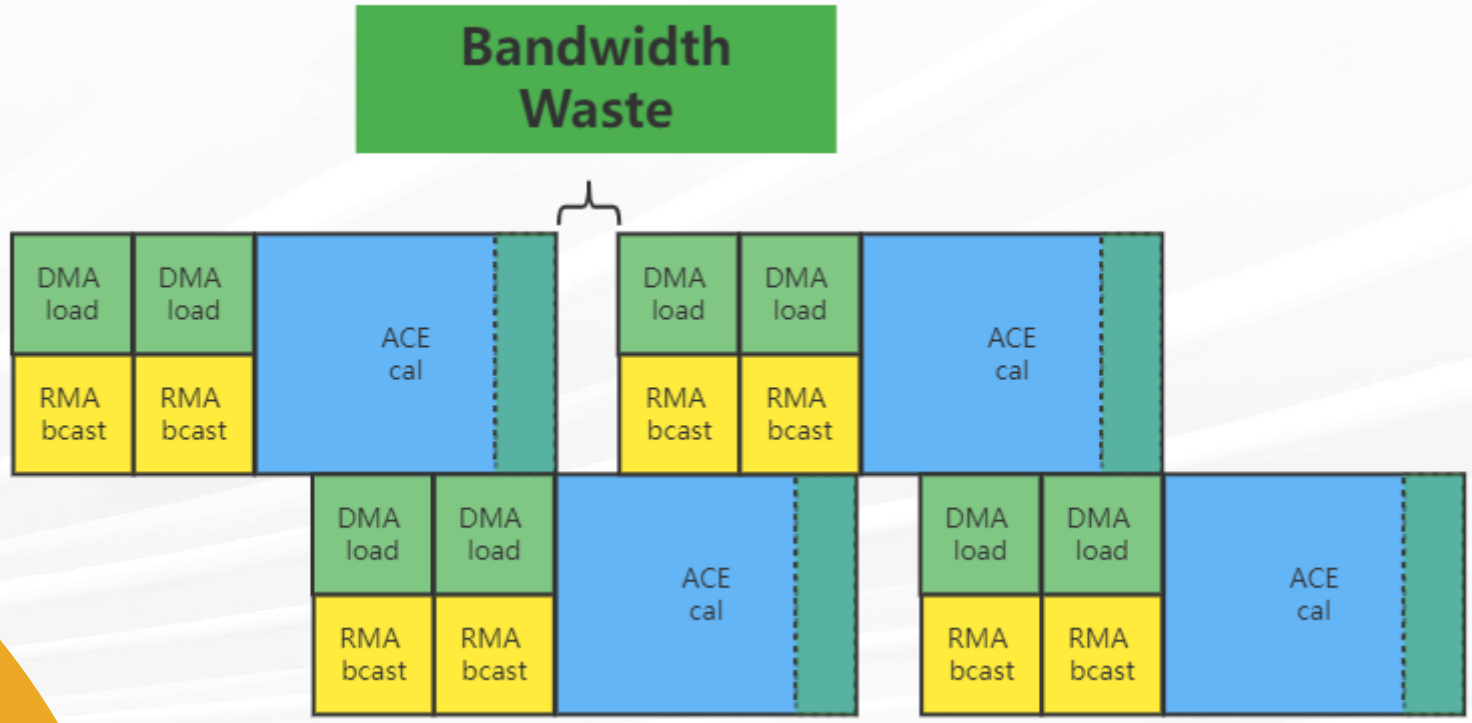
$$\begin{cases} T_{dma} < T_{ace} \text{ and } T_{rma} < T_{ace} \\ or \\ T_{dma} < T_{rma} \end{cases}$$

## 节省LDM

多核
GEMM

# SUMMA Variant

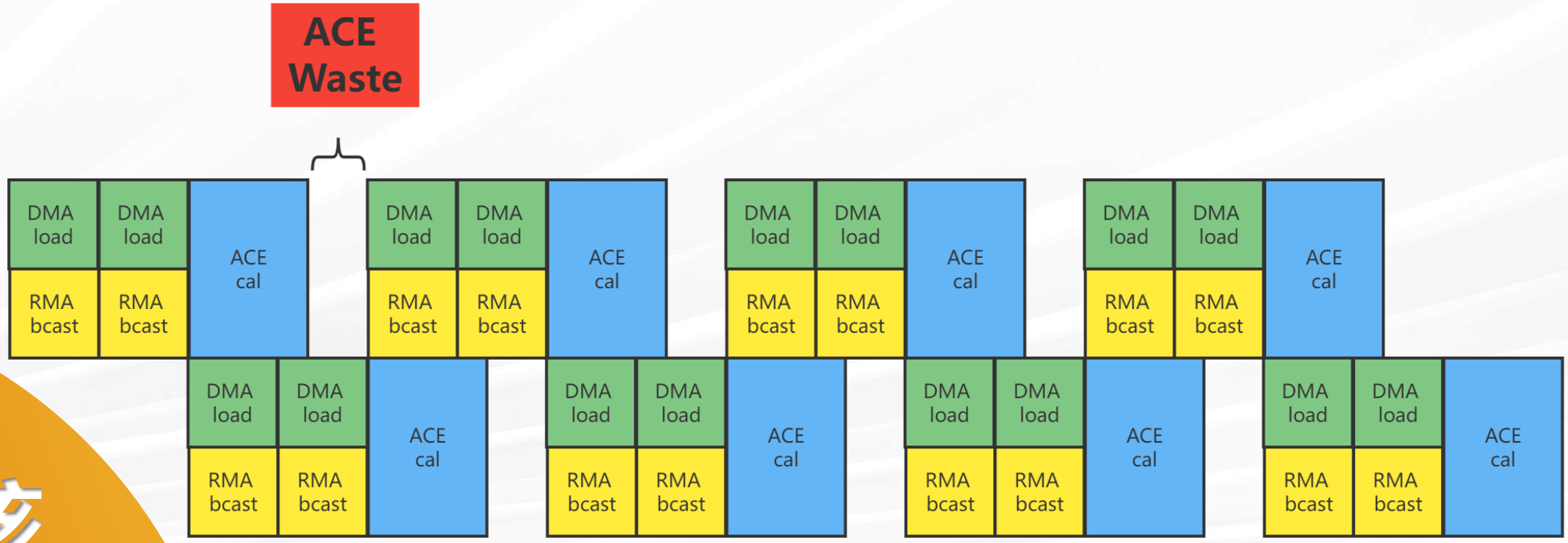
Tecorigin
太初元碁




多核
GEMM

# SUMMA Variant

Tecorigin
太初元碁




多核
GEMM

Tecorigin
太初元碁

4

# 复杂约束
--
# 细致分析

- ACE Shape约束
- DMA Shape约束
- D/RMA Conflict
- 最终结论

# DMA Shape约束

Tecorigin 太初元碁

1. 从核阵列在DMA访问主存的时候，首先地址会按照16个通道散开，每个通道 128Bytes，也就是说连续的128地址对齐的2KB数据访问请求（集合所有的DMA访问，不是单个从核的DMA访问），可以占满所有的通道，每个通道访问完全并行，通道利用率越高性能越好

```
int rt_dma_get_stride(
        void* dest,
        void* src,
        int len,
        int bsize,
        int stride)
```

bsize != 128 ?

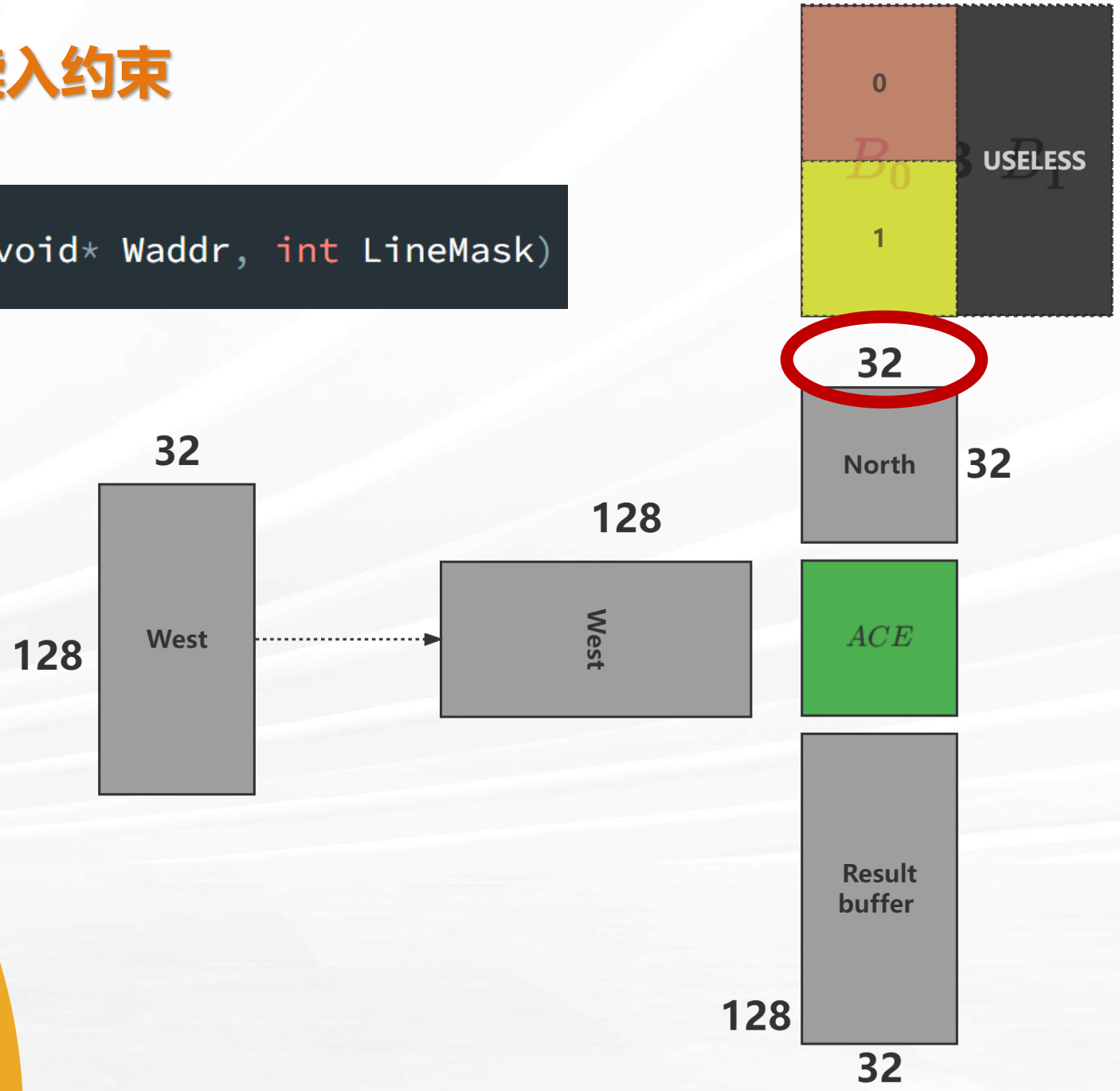
复杂约束

$$\text{Bandwidth}_{dma} = \text{Bandwidth}_{theory} * Discount_{dma}$$

# ACE Shape约束-读入约束

```
int rt_ace_load_north(void* Waddr, int LineMask)
```

**北向数据必须连续！**

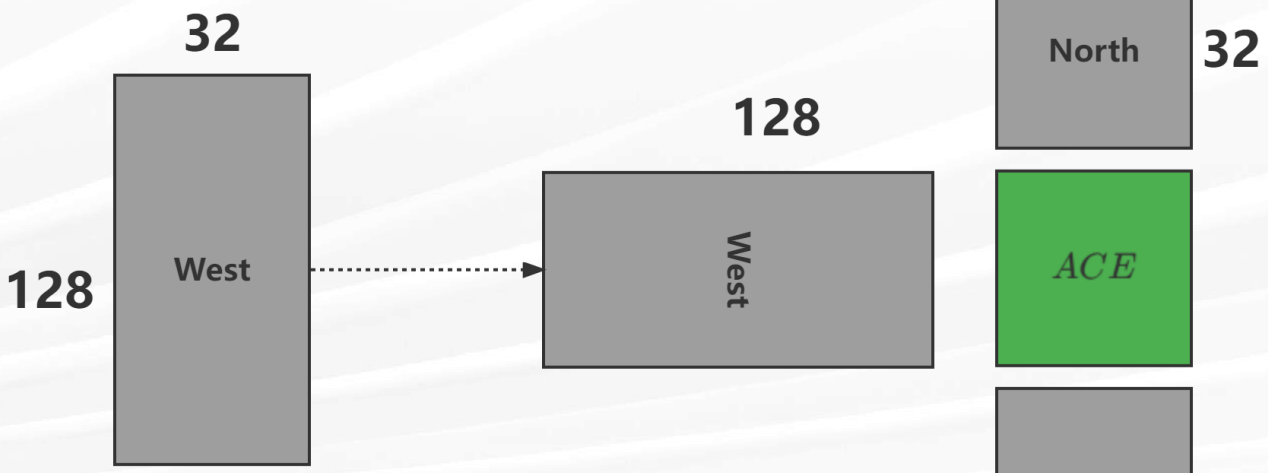


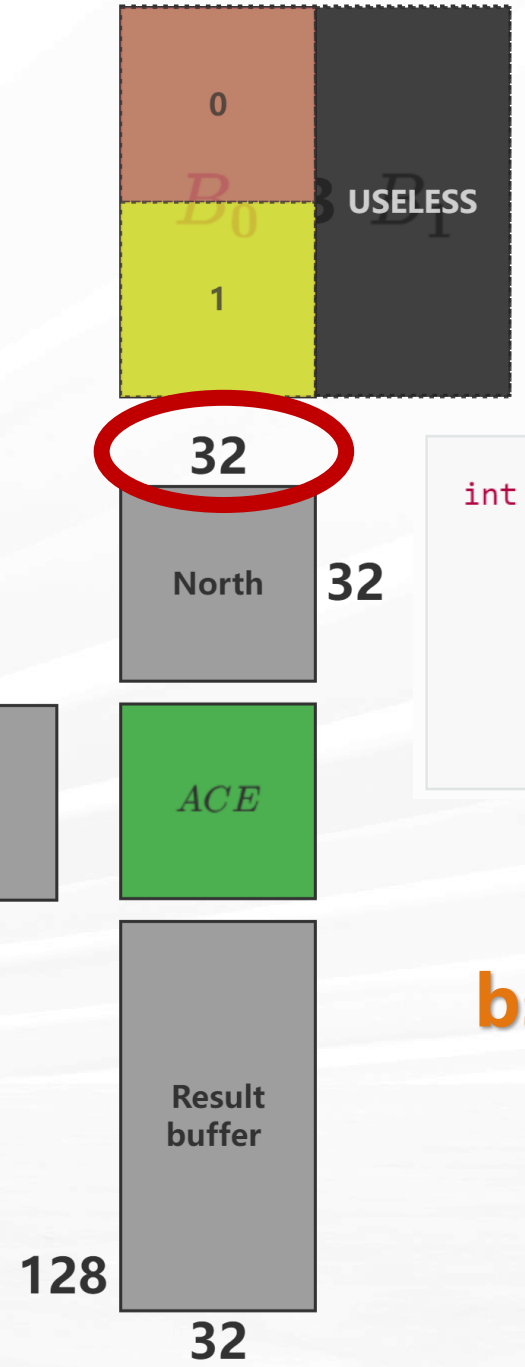

复杂约束

# ACE Shape约束-读入约束-DMA

```
int rt_ace_load_north(void* Waddr, int LineMask)
```

**北向数据必须连续！**




```
int rt_dma_get_stride(
        void* dest,
        void* src,
        int len,
        int bsize,
        int stride)
```

**bsize = 64**

**复杂约束**

# ACE Shape约束-读入约束-DMA

Tecorigin 太初元碁

1. 从核阵列在DMA访问主存的时候，首先地址会按照16个通道散开，每个通道 128Bytes，也就是说连续的128地址对齐的2KB数据访问请求（集合所有的DMA访问，不是单个从核的DMA访问），可以占满所有的通道，每个通道访问完全并行，通道利用率越高性能越好

```
int rt_dma_get_stride(
        void* dest,
        void* src,
        int len,
        int bsize,
        int stride)
```

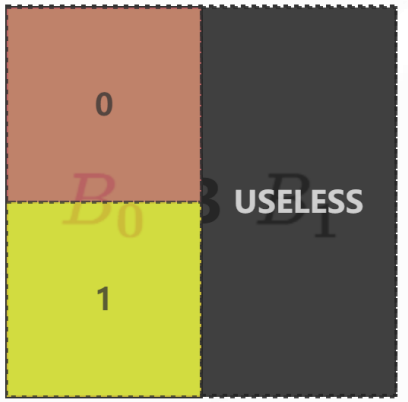
**bsize = 64 ?**

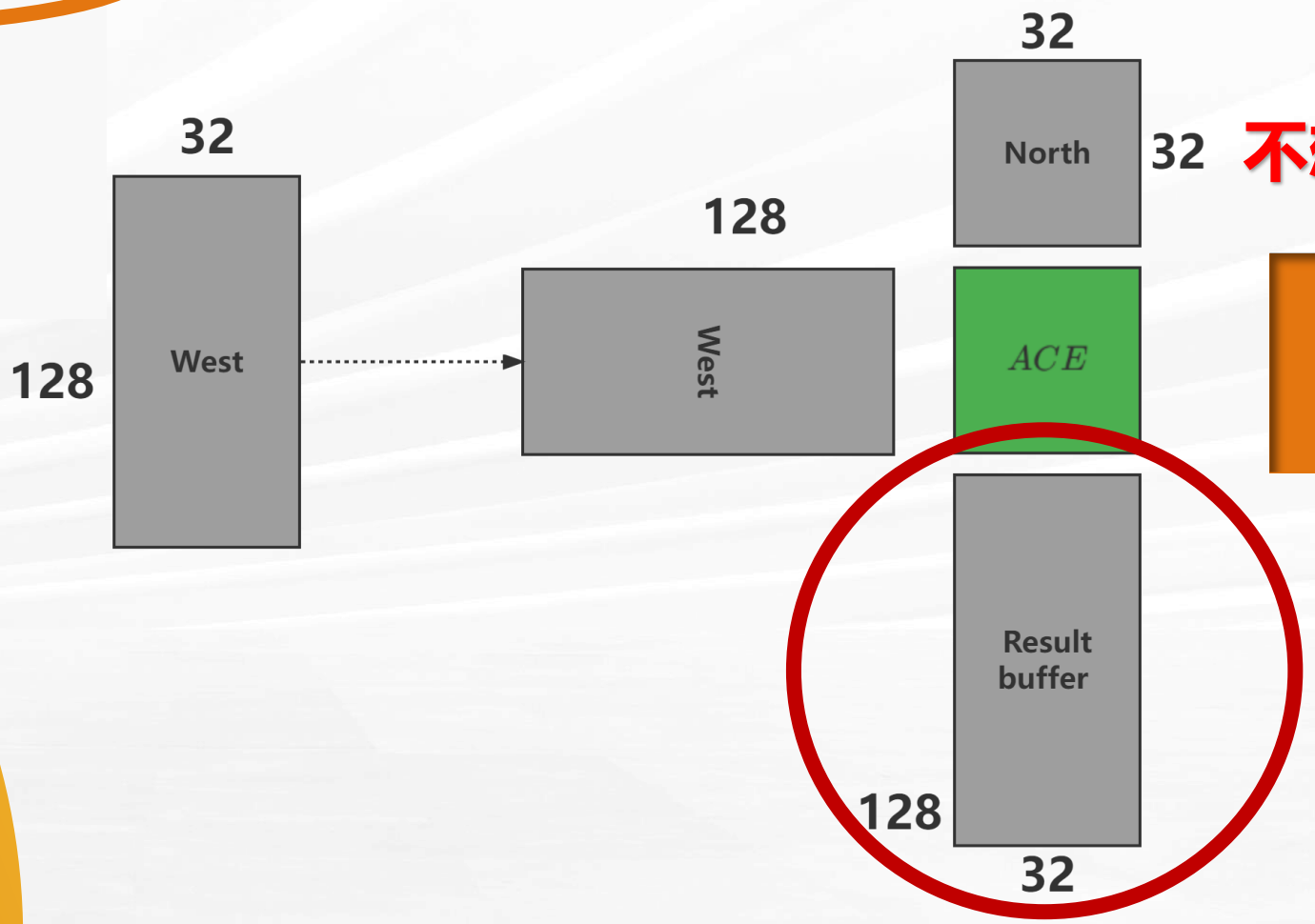
复杂约束

$$\text{Bandwidth}_{dma} = \text{Bandwidth}_{theory} * Discount_{dma}$$

# ACE Shape约束-写出约束

```
int rt_ace_load_west(
        void* Xaddr,
        void* Accumaddr,
        int Accumflag,
        int Xsize,
        int Loadstride,
        int Lastflag,
        int Precision,
        int Northflag)
```



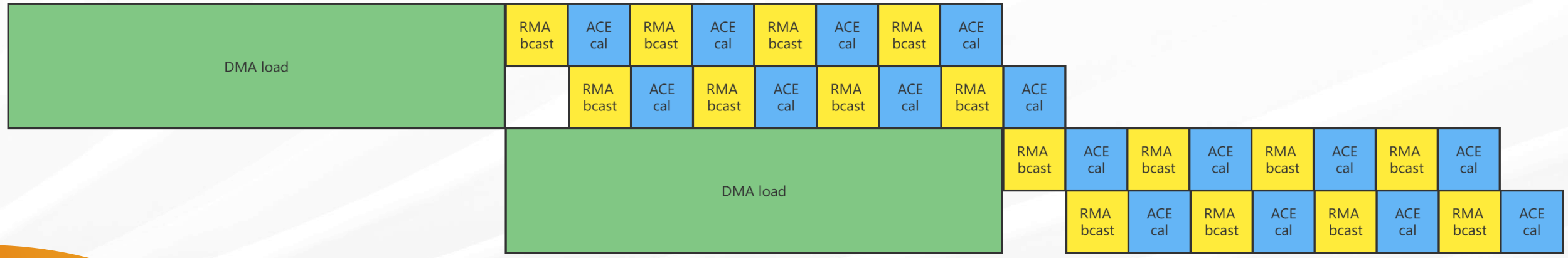




不想频繁ace_rtn?

$$n_{L2} * m_{L2} < 128 * 32$$

复杂约束

# ACE Shape约束-写出约束-RMA



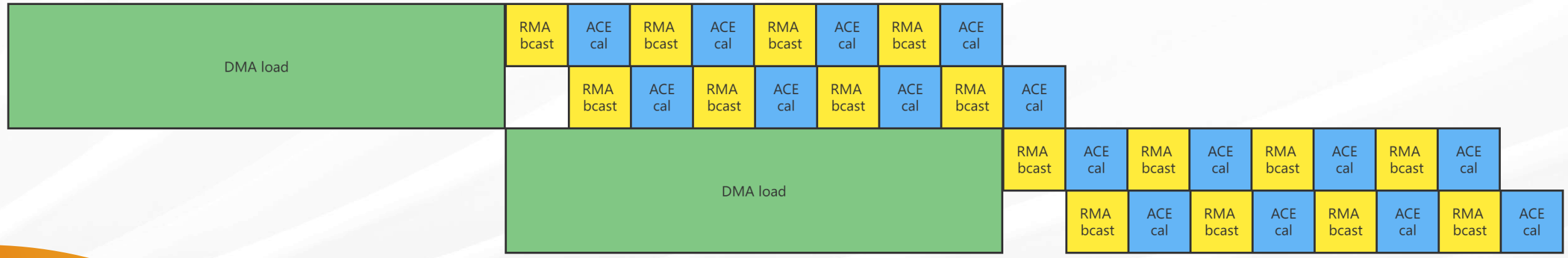

复杂约束

$$\begin{cases} \dfrac{n_{L1}m_{L1}}{n_{L1}+m_{L1}} > 320 \\ \dfrac{n_{L2}m_{L2}}{n_{L2}+m_{L2}} > 40 \end{cases}$$

?

$$n_{L2} * m_{L2} < 128 * 32$$

# ACE Shape约束-写出约束-RMA




复杂约束

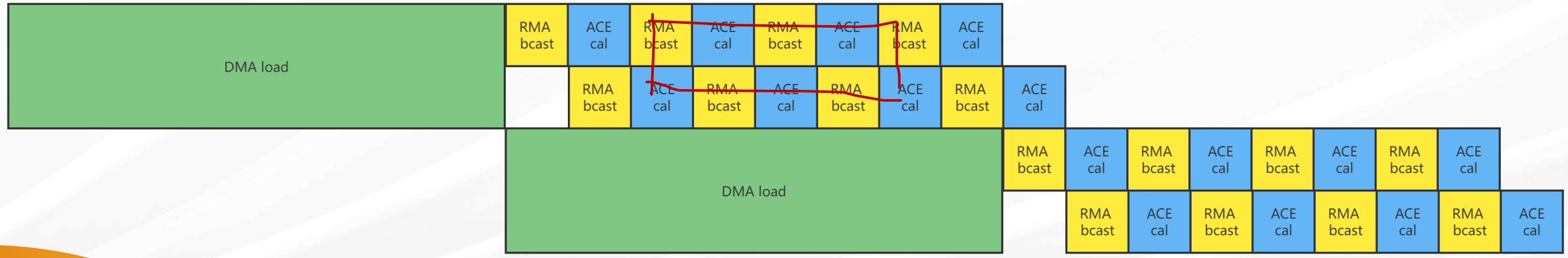
$$\begin{cases} \dfrac{n_{L1}m_{L1}}{n_{L1}+m_{L1}} > 320 \\ \dfrac{n_{L2}m_{L2}}{n_{L2}+m_{L2}} > 40 \end{cases}$$

?

$$n_{L2} * m_{L2} < 128 * 32$$

$$\frac{64 * 64}{64 + 64} > 40$$

# ACE Shape约束-写出约束-RMA



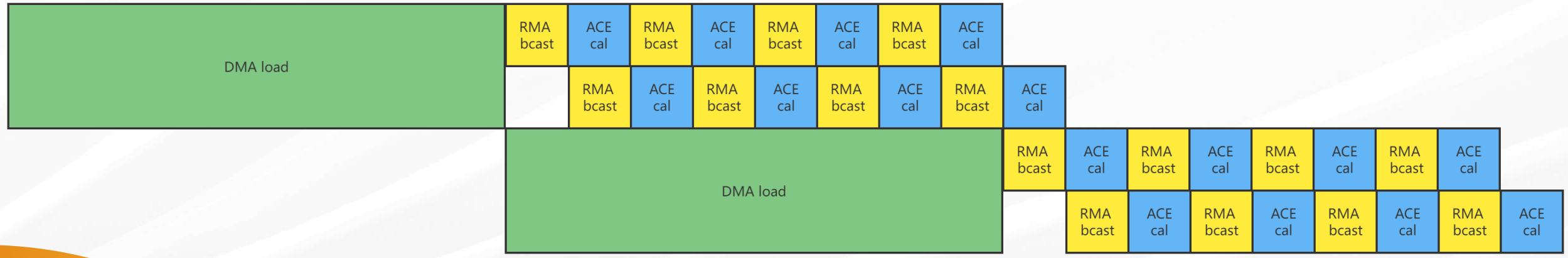

复杂约束

$$\begin{cases} \dfrac{n_{L1}m_{L1}}{n_{L1}+m_{L1}} > 320 \\ \dfrac{n_{L2}m_{L2}}{n_{L2}+m_{L2}} > 40 \end{cases}$$

$$n_{L2} * m_{L2} < 128 * 32$$

$$64 > 80$$ ✗

# ACE Shape约束-写出约束-RMA




复杂约束

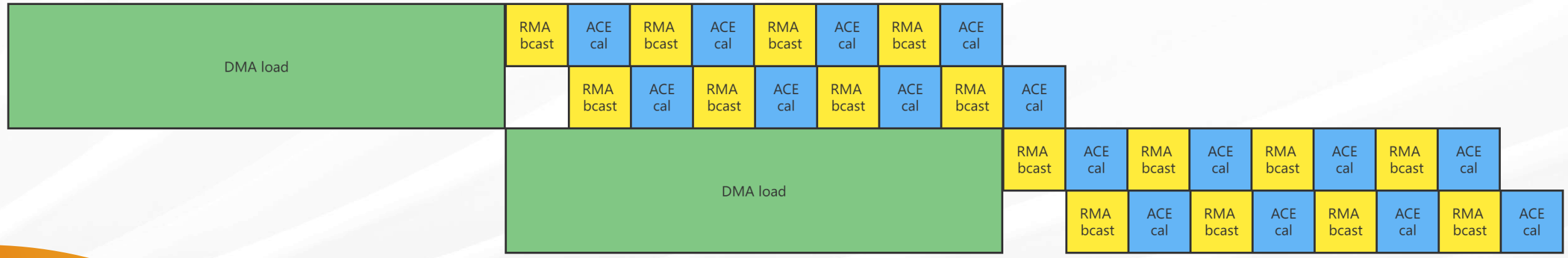
$$\begin{cases} \dfrac{n_{L1}m_{L1}}{n_{L1}+m_{L1}} > 320 \\ \dfrac{n_{L2}m_{L2}}{n_{L2}+m_{L2}} > 40 \end{cases}$$

$$n_{L2} * m_{L2} < 128 * 32$$

$$\begin{aligned} Performance &= Performance_{ace} * 64/80 \\ &= 64Tflops \end{aligned}$$

80%

# ACE Shape约束-写出约束-DMA




$$\begin{cases} \dfrac{n_{L1}m_{L1}}{n_{L1}+m_{L1}} > 320 \\ \dfrac{n_{L2}m_{L2}}{n_{L2}+m_{L2}} > 40 \end{cases} \quad \begin{cases} n_{L1} = 8n_{L2} \\ m_{L1} = 4m_{L2} \end{cases}$$

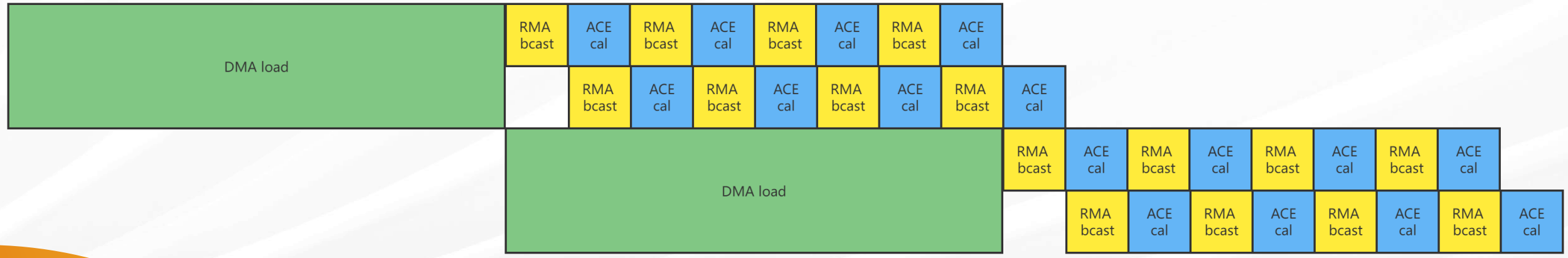
?

$$n_{L2} * m_{L2} < 128 * 32$$

复杂约束

$$\frac{64 * 64 * 8 * 4}{64 * 8 + 64 * 4} > 320$$

# ACE Shape约束-写出约束-DMA




复杂约束

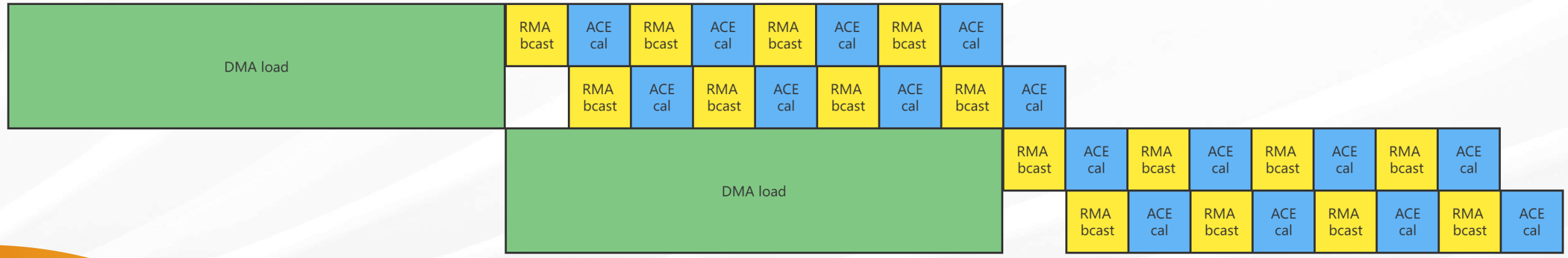
$$\begin{cases} \dfrac{n_{L1}m_{L1}}{n_{L1}+m_{L1}} > 320 \\ \dfrac{n_{L2}m_{L2}}{n_{L2}+m_{L2}} > 40 \end{cases} \quad \begin{cases} n_{L1} = 8n_{L2} \\ m_{L1} = 4m_{L2} \end{cases}$$

$$n_{L2} * m_{L2} < 128 * 32$$

$$171 > 320$$



# ACE Shape约束-写出约束-DMA




复杂约束

$$\begin{cases} \dfrac{n_{L1}m_{L1}}{n_{L1}+m_{L1}} > 320 \\ \dfrac{n_{L2}m_{L2}}{n_{L2}+m_{L2}} > 40 \end{cases} \quad \begin{cases} n_{L1} = 8n_{L2} \\ m_{L1} = 4m_{L2} \end{cases}$$

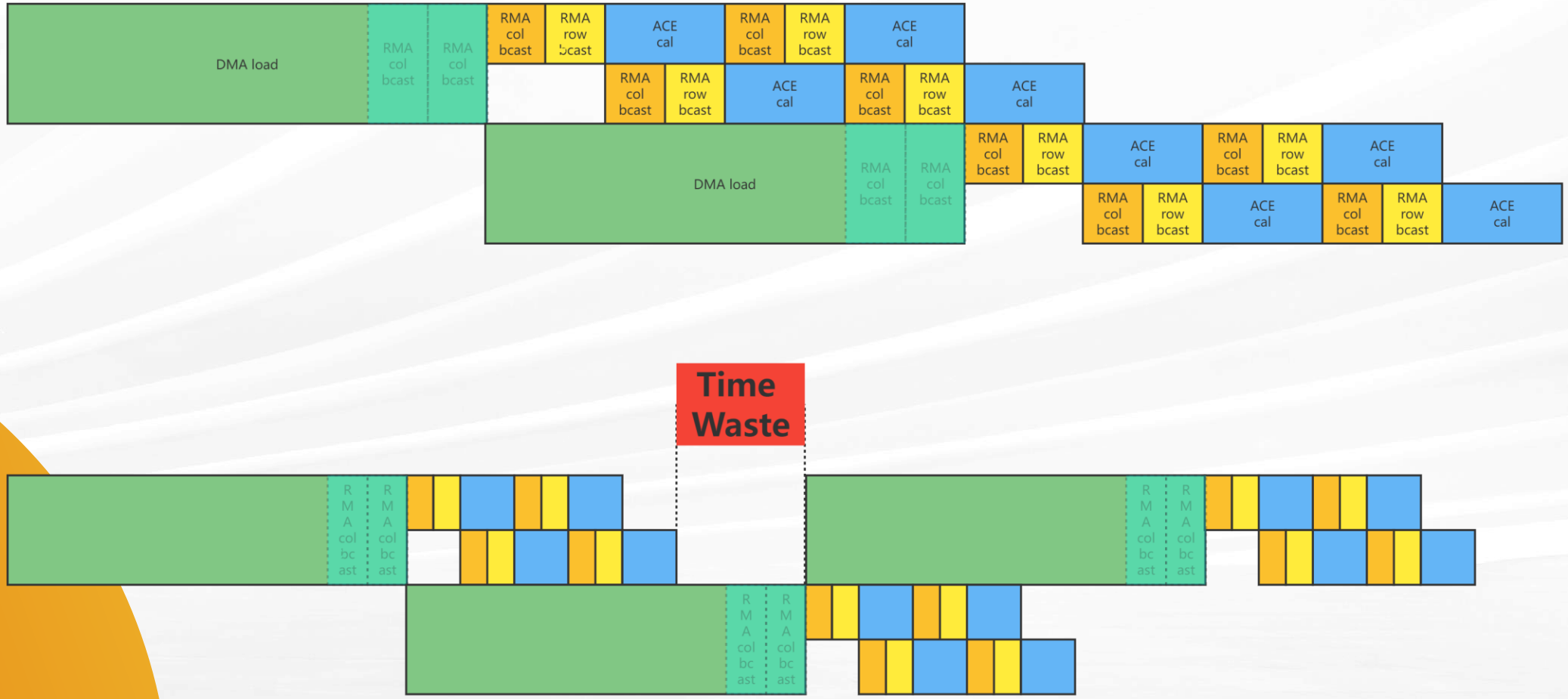
$$n_{L2} * m_{L2} < 128 * 32$$

$$\begin{aligned} Performance &= Performance_{ace} * 171/320 \\ &= 42.5Tflops \end{aligned}$$

53%

# DMA & RMA Conflict

Tecorigin
太初元碁




复杂约束

# 最终结论

Tecorigin
太初元碁

$$42.5Tflops * Discount_{dma}$$

复杂约束

# 最终结论

Tecorigin 太初元碁

$$42.5Tflops * Discount_{dma}$$

| $Discount_{dma}$ | $Performance\ (Tflops)$ |
|---|---|
| 100% | 42.5 |
| 90% | 38.3 |
| 80% | 34.0 |
| 70% | 29.7 |
| 60% | 25.5 |
| 50% | 21.3 |
| 40% | 17.0 |
| 30% | 12.8 |

复杂约束

Tecorigin
太初元碁

# Thanks.

铸造中国算力基石　构建未来智能世界

Where There is Computation, There is Tecorigin.

Tecorigin
太初元碁

# Q&A

铸造中国算力基石　构建未来智能世界

Where There is Computation, There is Tecorigin.

# 芯片性能表格

Tecorigin 太初元碁

| Key | Value | 实测值 | 备注 |
| --- | --- | --- | --- |
| Frequency | 2.5 G Hz | | |
| ACE(per CPE) | 2.5 T flops | 2.3 T flops | |
| ACE(CPE cluster) | 80 T flops | | |
| HBM_read | 250 GB/s | 250 GB/s | 非128B对齐时更低 |
| HBM_write | 280 GB/s | | |
| SIMD_Half(per CPE) | 80 G flops (double pipeline) | | |
| SIMD_Half(CPE cluster) | 2.5 T flops | | |
| RMA_get | 64 GB/s | 64 GB/s | |
| RMA_bcast | 64 GB/s | 52 GB/s | |
| DMA_bcast | 250 GB/s | 250 GB/s | |

Reference